Package 'squarebrackets'

July 9, 2026

Type Package

Title Subset Methods as Alternatives to the Square Brackets Operators for Programming

Version 0.0.0.9009

Description Provides subset methods
(supporting both atomic and recursive S3 classes)
that may be more convenient alternatives to the `[` and `[<-` operators,
whilst maintaining similar performance.
Some nice properties of these methods include, but are not limited to, the following.
1) The `[` and `[<-` operators use different rule-sets for different data.frame-like types
(data.frames, data.tables, tibbles, tidytables, etc.);
The 'squarebrackets' methods use the same rule-sets for the different data.frame-like types.
2) Performing dimensional subset operations on an array using `[` and `[<-`,
requires a-priori knowledge on the number of dimensions the array has;
The 'squarebrackets' methods work on any arbitrary dimensions without requiring such prior knowledge.
3) Unlike the `[` and `[<-` operators,
the 'squarebrackets' methods will operate on duplicate names
(not just the first match),
use consistent syntax for inverting indices,
and support the use of keywords through formulas.
4) The `[<-` operator only supports copy-on-modify semantics for most classes;
The 'squarebrackets' methods provides explicit pass-by-reference and pass-by-value semantics,
and does so safely.
5) 'squarebrackets' supports index-less sub-set operations for `long vectors`,
which is more memory efficient than sub-set operations using the `[` and `[<-` operators.

License MPL-2.0 | file LICENSE

Encoding UTF-8

LinkingTo Rcpp

Roxygen list(markdown = TRUE)

RoxygenNote 7.3.2

Suggests rlang,
stringi,
knitr,
rmarkdown,
tinytest,
tinycodet,
tidytable,

tibble,
ggplot2,
sf,
future.apply,
collections,
rrapply,
abind

Depends R (>= 4.2.0)

Imports methods,
Rcpp (>= 1.0.11),
collapse (>= 2.0.2),
data.table (>= 1.14.8)

URL https://github.com/tony-aw/squarebrackets/, https://tony-aw.github.io/squarebrackets/

BugReports https://github.com/tony-aw/squarebrackets/issues/

Language en-gb

Contents

`aaa00_squarebrackets_help`

squarebrackets: Subset Methods as Alternatives to the Square Brackets Operators for Programming

---

Description

squarebrackets:
Subset Methods as Alternatives to the Square Brackets Operators for Programming.

Provides subset methods (supporting both atomic and recursive S3 classes) that may be more convenient alternatives to the `[` and `[<-` operators, whilst maintaining similar performance.

Some nice properties of these methods include, but are not limited to, the following:

1. The `[` and `[<-` operators use different rule-sets for different data.frame-like types (data.frames, data.tables, tibbles, tidytables, etc.);
   The 'squarebrackets' methods use the same rule-sets for the different data.frame-like types.
2. Performing dimensional subset operations on an array using `[` and `[<-`, requires a-priori knowledge on the number of dimensions the array has;
   The 'squarebrackets' methods work on any arbitrary dimensions without requiring such prior knowledge.
3. Unlike the `[` and `[<-` operators, the 'squarebrackets' methods will operate on duplicate names (not just the first match), use consistent syntax for inverting indices, and support the use of keywords through formulas.
4. The `[<-` operator only supports copy-on-modify semantics for most classes;
   The 'squarebrackets' methods provides explicit pass-by-reference and pass-by-value semantics, and does so safely.
5. 'squarebrackets' supports index-less sub-set operations for `long vectors`, which is more memory efficient than sub-set operations using the `[` and `[<-` operators.

Goal

Among programming languages, 'R' has perhaps one of the most flexible and comprehensive sub-setting functionality, provided by the square brackets operators (`[`, `[<-`).
But in some situations the square brackets operators are occasionally less than optimally convenient

The Goal of the 'squarebrackets' package is not to replace the square-brackets operators, but to provide **alternative** sub-setting methods and functions, to be used in situations where the square bracket operators are inconvenient.

Quick Start Guide

For the Quick Start Guide, see:
https://tony-aw.github.io/squarebrackets/articles/squarebrackets.html.

Overview Help Pages

Essentials

The essential documentation is split into the following help pages:

- squarebrackets_methods:
  Lists the main methods provided by 'squarebrackets'.
  Also explains the method dispatch system in 'squarebrackets'.
- squarebrackets_index_fundamentals:
  Explains the essential fundamentals of the indexing forms in 'squarebrackets'.
- squarebrackets_keywords:
  Explains the usage of keywords in the main methods of 'squarebrackets'.

Arguments

The methods in 'squarebrackets' share a lot of common arguments.
The explanations for these common arguments are given in the following help pages:

- squarebrackets_supported_structures:
  Lists the structures that are supported by 'squarebrackets', and explains some related terminology.
- squarebrackets_index_args:
  Explains the common indexing arguments used in the main S3 methods.
- squarebrackets_modify:
  Explains the modification-related arguments, and other essential information regarding modification.
- squarebrackets_options:
  Lists and explains the options the user can specify in 'squarebrackets'.
- squarebrackets_stride:
  Gives an overview of the `stride` argument in the long_ methods.

Pass-By-Reference

The following help pages explain the pass-by-reference semantics provided by 'squarebrackets', and only need to be read when planning to use those semantics:

- squarebrackets_PassByReference:
  Explains Pass-by-Reference semantics, and its important consequences.
- squarebrackets_coercion:
  Explains the difference in coercion rules between modification through Pass-by-Reference semantics and modification through copy (i.e. pass-by-value).

Helper Functions

A couple of convenience functions, and helper functions for creating ranges, sequences, and indices (often needed in sub-setting) are provided:

- n: Nested version of c, and short-hand for list.
- ndim: Get the number of dimensions of an object.
- ss2coord, coord2ii: Convert subscripts (dimensional array indices) to coordinates, coordinates to flat indices, and vice-versa.
- match_all: Find all matches, of one vector in another, taking into account the order and any duplicate values of both vectors.
- Computing indices:
  idx_by to compute grouped indices.

Properties Details

The alternative sub-setting methods and functions provided by 'squarebrackets' have the following properties:

- **Programmatically friendly**:
  - Unlike base `[`, it's not required to know the number of dimensions of an array a-priori, to perform subset-operations on an array.
  - Missing arguments can be filled with `NULL`, instead of using dark magic like `base::quote(expr = )`.
  - No Non-standard evaluation.
  - Functions are pipe-friendly.
  - No (silent) vector recycling.
  - Extracting and removing subsets uses the same syntax.
- **Class consistent**:
  - sub-setting of multi-dimensional objects by specifying dimensions (i.e. rows, columns, ...) use `drop = FALSE`.
    So matrix in, matrix out.
  - The methods deliver the same results for data.frames, data.tables, tibbles, and tidytables. No longer does one have to re-learn the different brackets-based sub-setting rules for different types of data.frame-like objects.
    Powered by the subclass agnostic 'C'-code from 'collapse' and 'data.table'.
- **Explicit copy semantics**:
  - Sub-set operations that change its memory allocations, always return a modified (partial) copy of the object.
  - For sub-set operations that just change values in-place (similar to the `[<-` and `[[<-` methods) the user can choose a method that modifies the object by **reference**, or choose a method that returns a **(partial) copy**.
- **Careful handling of names**:
  - Sub-setting an object by index names returns ALL matches with the given names, not just the first.
  - Data.frame-like objects (see supported classes below) are forced to have unique column names.

- **Concise function and argument names**.
- **Performance & Energy aware**:
  Despite the many checks performed, the functions are kept reasonably speedy, through the use of the 'Rcpp', 'collapse', and 'data.table' R-packages.
  The functions were also made to be as memory efficient as reasonably possible, to lower the carbon footprint of this package.

Author(s)

Author, Maintainer: Tony Wilkes <tony_a_wilkes@outlook.com> (ORCID)

References

The badges shown in the documentation of this R-package were made using the services of: https://shields.io/

`aaa01_squarebrackets_methods`
Methods Available in 'squarebrackets'

Description

This help page gives an overview of the methods available in 'squarebrackets'.

Main Methods

The main methods of 'squarebrackets' use the naming convention `<indexform>_<operation>`:
`<indexform>` tells you what form of indices the method uses;
`<operation>` tells you **what operation** is performed.

For the `<indexform>` part, the following is available:

- `ii_`: operates on subsets of atomic/recursive vectors/arrays by interior indices.
- `ss_`: operates on subsets of atomic/recursive arrays of any dimensionality by subscripts.
- `tt_`: operates on subsets of data.frames and atomic/recursive matrices by tabular indices (also known as tabulat tiles).

For the `<operation>` part, the following is available:

- `_x`: extract, exchange, exclude, or duplicate (if applicable) subsets.
- `_mod`: modify subsets and return copy.
- `_set`: modify subsets using pass-by-reference semantics.
- `icom`: create indices to be used in `[<-` to modify objects using the default Copy-On-Modify semantics.

To illustrate, let's take the methods used for extracting subsets (_x):

- If y is a vector or array (of any dimension),
  ii_x(y, i) corresponds to y[i].
- If y is a 3d array,
  ss_x(y, n(i, k), c(1, 3)) corresponds to y[i, , k, drop = FALSE].
- If y is a matrix or data.frame-like object,
  tt_x(y, i, j) corresponds to y[i, j, drop = FALSE].

Specialized Methods

The main methods of 'squarebrackets' are applicable for all supported types and classes (provided the correct method for the correct dimensionality is used).
'squarebrackets' also provides specialized methods specific to certain structures:

- the lst_ set of methods, which deal with sub-set operations that are only relevant for (nested) lists, but not for the other types of supported objects.

- the long_ set of methods, which deal with index-less sub-set operations, that are only relevant for long atomic vectors.

Other Methods

Besides the previously mentioned methods, 'squarebrackets' provides some additional methods that do not neatly fit into the above methods.
These are the set of sb_ methods, which cover miscellaneous operations for atomic/recursive objects.

Finding the Appropriate Help Pages

With knowledge of the naming convention of the main methods, one can easily find out information about a particular method by usign the ? operator.
So to find out about modifying objects by subscripts using Pass-by-Reference semantics, type in:
?ss_set

'squarebrackets' Methods that do not require explicit Dispatches

The ii_/ss_/tt_ - _x methods are essentially wrappers around the [operator.
Similarly, the ii_/ss_/tt_ - _mod methods are essentially wrappers around the [<- operator.
And the lst_rec and lst_recin methods are essentially wrappers around the [[and [[<- operators.
Therefore, any custom class that has method dispatches defined for the [, [<-, [[, and [[<- methods, have their dispatches automatically handled by the above named methods.

'squarebrackets' Methods that DO require explicit Dispatches

Unlike the `ii_/ss_/tt_ - _x` methods, the long_x method is **not** a wrapper around the `[` operator, and thus does not detect custom method dispatches defined for `[`.
long_x can support classes that have static attributes (attributes that do not depend on the type, length, or data of the vector), by adding the class names to the squarebrackets.sticky option.
Classes with non-static attributes will explicitly require a `long_x` method.

Class support for the Pass-By-Reference Methods

The `_set` methods only support the mutable classes `data.table` and `mutatomic`.
Other mutable classes are not supported by the 'squarebrackets' package.
However, other 'R' package authors are welcome to add additional method dispatches for the `_set` methods.

`aaa02_squarebrackets_index_fundamentals`
Indexing Fundamentals of 'squarebrackets'

Description

This help page explains the fundamentals regarding how 'squarebrackets' treats indexing.
Some familiarity with base R's `[` and `[<-` operators is required to follow this help page.

Indexing Forms

Consider the following representation of array indices for a (in this case) 4 by 5 matrix:

```
     [,1] [,2] [,3] [,4] [,5]
[1,] [1]  [5]  [9]  [13] [17]
[2,] [2]  [6]  [10] [14] [18]
[3,] [3]  [7]  [11] [15] [19]
[4,] [4]  [8]  [12] [16] [20]
```

The numbers `1` to `20` on the interior of this representation, are referred to in this documentation as "interior indices" (abbreviated as "ii"), also known as "flat indices".
The numbers on the edges of this representations, `1` to `4` for the rows and `1` to `5` for the columns, are referred to in this documentation as "subscripts" (abbreviated as "ss"), also known as "dimensional indices".
Indexing by rows and columns, referred to as tabular tiles (abbrevated as "tt"), is a commonly used special subset of using subscripts, available only for data.frames and matrices.

Thus 'squarebracets' supports these 3 forms of indexing:
Indexing by interior indices, indexing by subscripts, and tabular tiles.

Regarding which kind of object supports which kind of indexing form:

- Matrices, which are simply 2-dimensional arrays, support all 3 of the above given indexing forms.
- Arrays in general can always support both interior indices and subscripts.
- Dimensionless vectors (i.e. objects for which `dim()` returns `NULL`) only support interior indices.
- Data.frames only support tabular tiles.

Regarding which set of methods support which kind of indexing form:

- One can operate on flat/interior indices (often simply referred to as "indices") using the ii_ methods.
  These primarily use the i, use argument pair.
  One can also use the long_ methods, though these operate on **virtual** interior indices called a stride.
- One can operate on general subscripts (= dimensional indices) using the ss_ methods;
  These primarily use the the s, use argument pair.
- One can operate on tabular tiles using the tt_ methods;
  These primarily use the the row, col, use argument pair.

For the relationship between flat/interior indices and subscripts for arrays, see the ss2ii help page.

Indexing Types

Base 'R' supports indexing through `logical`, `integer`, and `character` vectors.
'squarebrackets' supports these also (albeit with some improvements), but also supports some additional methods of indexing.

Whole numbers

Whole numbers are the most basic form on index selection.
All forms of indexing in 'squarebrackets' are internally translated to integer (or double if `> (2^31 - 1)`) indexing first, ensuring consistency.
Indexing through integer/numeric indices in 'squarebrackets' works the same as in base 'R', except that negative values are not allowed.
So indexing starts at `1` and is inclusive.

Logical

Selecting indices with a logical vector in 'squarebrackets' works the same as in base 'R', except that recycling is not allowed.

Characters

When selecting indices using a character vector, base 'R' only selects the first matches in the names.
'squarebrackets', however, selects all matches:

```
nms <- c("a", letters[4:1], letters[1:5])
x <- 1:10
names(x) <- nms
print(x) #' `x` has multiple elements with the name "a"
#>  a  d  c  b  a  a  b  c  d  e
#>  1  2  3  4  5  6  7  8  9 10

ii_x(x, "a") # extracts all indices with the name "a"
#> a a a
#> 1 5 6

ii_x(x, c("a", "a")) # repeats all indices with the name "a"
#> a a a a a a
#> 1 5 6 1 5 6
```

Character indices are internally translated to integer indices using match_all.

Inverting

Inverting indices means to specify all elements **except** the given indices.
Consider for example the atomic vector `month.abb` (abbreviate month names).
Given this vector, indices `1:5` gives `c("Jan" "Feb" "Mar" "Apr", "May")`.
Inverting those same indices will give `c("Jun" "Jul" "Aug" "Sep" "Oct" "Nov" "Dec")`.

In base 'R', inverting an index is done in different ways.
(negative numbers for numeric indexing, negation for logical indexing, manually un-matching for character vectors).

In 'squarebrackets', inverting is consistently done through the `use` argument:
A positive sign for `use` means to select the specified indices, a negative sign for `use` means to select all indices **except** the specified indices.

EXAMPLES

```
x <- month.abb
print(x)
#>  [1] "Jan" "Feb" "Mar" "Apr" "May" "Jun" "Jul" "Aug" "Sep" "Oct" "Nov" "Dec"


ii_x(x, 1:5) # extract first 5 elements
#> [1] "Jan" "Feb" "Mar" "Apr" "May"

ii_x(x, 1:5, -1) # return WITHOUT first 5 elements
#> [1] "Jun" "Jul" "Aug" "Sep" "Oct" "Nov" "Dec"


ii_mod(x, 1:5, rp = "XXX") # copy, replace first 5 elements, return result
#>  [1] "XXX" "XXX" "XXX" "XXX" "XXX" "Jun" "Jul" "Aug" "Sep" "Oct" "Nov" "Dec"
```

```
ii_mod(x, 1:5, -1, rp = "XXX") # same, but for all except first 5 elements
#>  [1] "Jan" "Feb" "Mar" "Apr" "May" "XXX" "XXX" "XXX" "XXX" "XXX" "XXX" "XXX"
```

ABOUT ORDERING
The order in which the user gives indices when inverting indices generally does not matter.
The order of the indices as they appear in the original object `x` is maintained, just like in base 'R'.

Out-of-Bounds Integers, Non-Existing Names, and NAs

- Integer indices that are out of bounds (including `NaN` and `NA_integer_`) always give an error.
- Character indices that specify non-existing names is considered a form of zero-length indexing.
  Specifying `NA` names returns an error.
- Logical indices are translated internally to integers using which, and so `NA`s are ignored.

Index-less Sub-set Operations

Until now this help page focussed on performing sub-set operations with an indexing vector.

Performing sub-set operations on a long vector using a index vector (which may itself also be a long vector) is not very memory-efficient.
'squarebrackets' therefore introduces index-less sub-set operations, through the long_ methods.
These methods are much more memory and computationally efficient than index-based sub-set methods (and so also a bit better for the environment!).

Regarding Performance

Integer vectors created through the `:` operator are "compact ALTREP" integer vectors, and provide the fastest way to specify indices.
Indexing through names (i.e. character vectors) is the slowest.

Index-less sub-set operations are usually faster and more memory efficient than any index-based sub-set operation.
So if performance is important, use index-less sub-set operations, or use compact ALTREP integer indices.

Indexing in Recursive Subsets

Until now this help page focussed on indexing for regular (or "shallow") subsets.
This section will discuss indexing in recursive subsets.

One of the differences between atomic and recursive objects, is that recursive objects support recursive subsets, while atomic objects do not.

Bear in mind that every element in a recursive object is a reference to another object.
Consider the following list `x`:

```
x <- list(
   A = 1:10,
   B = letters,
   C = list(A = 11:20, B = month.abb)
)
```

Regular subsets, AKA surface-level subset operations (`[`, `[<-` in base 'R'), operate on the recursive object itself.
I.e. ii_x(`x, 1`), or equivalently `x[1]`, returns the **list** `list(A = 1:10)`:

```
ii_x(x, 1) # equivalent to x[1]; returns list(A = 1:10)
#> $A
#>  [1]  1  2  3  4  5  6  7  8  9 10
```

Recursive subset operations (`[[`, `[[<-`, and `$` in base 'R'), on the other hand, operate on an object a subset of the recursive object references to.
I.e. lst_rec(`x, 1`), or equivalently `x[[1]]`, returns the **integer vector** `1:10`:

```
lst_rec(x, 1) # equivalent to x[[1]]; returns 1:10
#>  [1]  1  2  3  4  5  6  7  8  9 10
```

Recursive objects can refer to other recursive objects, which can themselves refer to recursive objects, and so on.
Recursive subsets can go however deep you want.
So, for example, to extract the character vector `month.abb` from the aforementioned list `x`, one would need to do:
lst_rec(`x, c("C","B")`), (in base R: `x$C$B`):

```
lst_rec(x, c("C","B")) # equivalent to x$C$B
#>  [1] "Jan" "Feb" "Mar" "Apr" "May" "Jun" "Jul" "Aug" "Sep" "Oct" "Nov" "Dec"

# or:

lst_rec(x, c(3, 2)) # equivalent to x[[3]][[2]]
#>  [1] "Jan" "Feb" "Mar" "Apr" "May" "Jun" "Jul" "Aug" "Sep" "Oct" "Nov" "Dec"
```

LIMITATIONS
Indexing in recursive subsets is significantly more limited than in regular (or "shallow") subsets:

- Recursive subset operations using lst_rec/lst_recin only support positive integer vectors and character vectors.
- Logical vectors are not supported.
- Since a recursive subset operation only operates on a single element, specifying the index with a character vector only selects the first matching element (just like base 'R'), not all matches.

- Inverting indices is also **not** available for recursive indexing.
- Unlike regular sub-setting, out-of-bounds specification for indices is acceptable, as it can be used to add new values to lists.

Non-Standard Evaluation

'squarebrackets' is designed primarily for programming, and seeks to be fully programmatically friendly.
As part of this endeavour, 'squarebrackets' never uses Non-Standard Evaluation.
All input for all methods and functions in 'squarebrackets' are objects that can be stored in a variable.
Like atomic vectors, lists, formulas, etc.

`aaa03_squarebrackets_supported_structures`
Supported Structures

Description

'squarebrackets' only supports the most common S3 objects, and only those that primarily use square brackets for sub-set operations (hence the name of the package).

One can generally divide the structures supported by 'squarebrackets' along 3 key properties:

- atomic vs recursive:
  Types `logical`, `integer`, `double`, `complex`, `character`, and `raw` are `atomic`.
  Lists and data.frames are `recursive`.
- dimensionality:
  Whether an object is a vector, array, or data.frame.
  Note that a matrix is simply an array with 2 dimensions.
- mutability:
  Base R's S3 classes (except Environments) are generally immutable:
  Modifying the object will create a copy (called 'copy-on-modify').
  'squarebrackets also supports `data.tables` and mutatomic objects, which are mutable:
  If desired, one can modify them without copy using pass-by-reference semantics.

Supported Structures

'squarebrackets' supports the following immutable structures:

- basic `atomic` classes
  (atomic vectors and arrays).
- factor.

- basic list classes
  (recursive vectors and arrays).

- data.frame
  (including the classes `tibble`, `sf-data.frame` and `sf-tibble`).

'squarebrackets' supports the following mutable structures:

- mutatomic
  (`mutatomic` vectors and arrays);
- data.table
  (including the classes `tidytable`, `sf-data.table`, and `sf-tidytable`).

The methods provided by 'squarebrackets', like any method, can be extended (by other 'R' package authors) to support additional classes that are not already supported natively by 'squarebrackets'.

Details

Atomic vs Recursive

Atomic and recursive objects are quite different from each other in some ways:

- **homo- or heterogeneous**: an atomic object can only have values of one data type.
  recursive objects can hold values of any combination of data types.
- **nesting**: Recursive objects can be nested, while atomic objects cannot be nested.
- **copy and coercion effect**: One can coerce or copy a subset of a recursive object, without copying the rest of the object. For atomic objects, however, a coercion or copy operation coerces or copies the entire vector (ignoring attributes).
- **vectorization**: most vectorized operations generally work on atomic objects, whereas recursive objects often require loops or apply-like functions.
  Therefore, when transforming recursive objects using the `*_mod` and `*_set` methods, 'squarebrackets' will apply transformation via lapply.
- **recursive subsets**: Recursive objects distinguish between "regular" subset operations (in base R using `[`, `[<-`), and recursive subset operations (in base R using `[[`, `[[<-`).
  See the lst_rec method and the dropl helper function.
  For atomic objects, these 2 have no meaningful difference (safe for perhaps some minor attribute handling).

- **views**: For recursive objects, one can create a View of a recursive subset (see also square-brackets_coercion).
  Subset views do not exist for atomic objects.

Dimensionality

'squarebrackets' supports dimensionless or vector objects (i.e. ndim `== 0L`).
squarebrackets' supports arrays (see is.array and is.matrix); note that a matrix is simply an array

with 2 dimensions.
'squarebrackets' also supports data.frame-like objects (see is.data.frame).
Specifically, squarebrackets' supports a wide variety of data.frame classes:
`data.frame`, `data.table`, `tibble`, `tidytable`;
'squarebrackets' also supports their 'sf'-package compatible counter-parts:
`sf-data.frame`, `sf-data.table`, `sf-tibble`, `sf-tidytable`.

Dimensionless vectors and dimensional arrays are supported in both their atomic and recursive forms.
Data.frame-like objects, in contrast, only exist in the recursive form (and, as stated, are supported by 'squarebrackets').
Recursive vectors, recursive matrices, and recursive arrays, are collectively referred to as "lists" in the 'squarebrackets' documentation.

Note that the dimensionality of data.frame-like objects is not the same as the dimensionality of (recursive) arrays/matrices.
For example:
For any array/matrix `x`, it holds that `length(x) == prod(dim(x))`.
But for any data.frame `x`, it is the case that `length(x) == ncol(x)`.

Mutable vs Immutable
Most of base R's S3 classes (except Environments) are generally immutable:
Modifying the object will create a copy (called 'copy-on-modify').
They have no explicit pass-by-reference semantics.

Supported mutable structures:

- 'squarebrackets' supports the mutable `data.table` class
  (and thus also `tidytable`, which inherits from `data.table`).
- 'squarebrackets' supports the mutatomic class.
  mutatomic objects are the same as atomic objects, except they are mutable (hence the name).

Supported immutable structures:
Atomic and recursive vectors/matrices/arrays, data.frames, and tibbles.

All the functions in the 'squarebrackets' package with the word "set" in their name perform pass-by-reference modification, and thus only work on mutable structures.
All other functions work the same way for both mutable and immutable structures.

Derived Atomic Vector
A special class of objects are the Derived Atomic Vector structures:
structures that are derived from atomic objects, but behave differently.
For example:
Factors, datetime, POSIXct and so on are derived from atomic vectors.
But they have attributes and special methods that make them behave differently.

'squarebrackets' treats derived atomic classes as regular atomic vectors.
There are highly specialized packages to handle objects derived from atomic objects.

For example, the 'anytime' package to handle date-time objects.

'squarebrackets does provide some more explicit support for factors.

Not Supported S3 structures
Key-Values storage S3 structures, such as environments, are not supported by 'squarebrackets'.

`aaa04_squarebrackets_index_args`
Index Arguments in the Generic Sub-setting Methods

Description

There are several types of arguments that can be used in the generic methods of 'squarebrackets' to specify the indices to perform operations on:

- `i, use`: to specify interior (i.e. dimensionless) indices.
- `s, use`: to specify subscripts of arbitrary dimensions in arrays.
- `row, col, use`: to specify rows and columns in tabular objects (matrices and data.frames).
- `slice, use`: to specify indices of one particular dimension (for arrays and data.frame-like objects).
  Currently only used in the `_icom` methods.

For the fundamentals of indexing in 'squarebrackets', see squarebrackets_index_fundamentals.
In this help page `x` refers to the object on which subset operations are performed.

Argument Pair `i, use`

class: atomic vector
class: recursive vector
class: atomic array
class: recursive array

`i` specifies the interior (or flat) indices.
`use` can be `1` or `-1`:
`1` means the specified indices in `i` are to be used for the sub-set operation.
`-1L` means **all** indices **except** the indices in `i` are to be used for the sub-set operation.

Any of the following can be specified for argument `i`:

- `NULL` or `0L`, corresponds to missing argument.

- a vector of length 0, in which case no indices are selected for the operation (i.e. empty selection).
- a numeric vector of **strictly positive whole numbers** giving indices.
- a **logical vector**, of the same length as `x`, giving the indices to select for the operation.
- a **character** vector of index names.
  If an object has multiple indices with the given name, ALL the corresponding indices will be selected for the operation.
- a **function** that takes as input `x`, and returns a logical vector, giving the element indices to select for the operation.
  For atomic objects, `i` is interpreted as `i(x)`.
  For recursive objects, `i` is interpreted as `lapply(x, i)`.
- a formula; see keywords.

Using the `i` arguments corresponds to doing something like the following:

```
ii_x(x, i = i) # ==> x[i]   # if `x` is atomic
ii_x(x, i = i) # ==> x[i]  # if `x` is recursive
```

If `i` is a function, it corresponds to the following:

```
ii_x(x, i = i) # ==> x[i(x)] # if `x` is atomic
ii_x(x, i = i) # ==> x[lapply(x, i)] # if `x` is recursive
```

Argument Pair `s, use`

class: atomic array
class: recursive array
The `s, use` argument pair, inspired by the `abind::asub` function from the 'abind' package, is the primary indexing argument for sub-set operations on atomic arrays or recursive arrays.

The `s` argument specifies the **subscripts** (i.e. dimensional indices).
The **absolute value** of `use` argument gives the dimensions for which the `s` holds (i.e. `use` specifies the "non-missing" margins), and the sign (+ or -) of `use` specifies if the indices are to be selected or excluded, similar to the `use` argument used in combination with `i` (see previous section).

Specifically, the `use` argument can be any of the following:

- a vector of length `0`, which will be interpreted as "all subscripts are missing".
- an integer vector.
- the real scalar `Inf`, which will be interpreted as `1:ndim(x)`.
- the real scalar `-Inf`, which will be interpreted as `-1:-ndim(x)`.

`use` is not allowed to have any duplicate values, nor is it allowed to include zero (`0`).
The default value for `use` is **lazy evaluated** `1:ndim(x)`.

`s` must be an atomic vector, a list of length 1, or a list of the same length as `use`.
If `s` is a list of length 1, it is internally recycled to become the same length as `use`.
If `s` is an atomic vector, it is internally treated as `list(s)`, and (as with the previous case) recycled to become the same length as `use`.

Each element of `s` when `s` is a list, or `s` as a whole when `s` is atomic, can be any of the following:

- `NULL` or `0L`, which corresponds to a missing index argument.
- a vector of length 0, in which case no indices are selected for the operation (i.e. empty selection).
- a numeric vector of **strictly positive whole numbers** with indices of the specified dimension to select for the operation.
- a **logical** vector of the same length as the corresponding dimension size, giving the indices of the specified dimension to select for the operation.
- a **character** vector giving the `dimnames` to select.
  If a dimension has multiple indices with the given name, ALL the corresponding indices will be selected for the operation.
- a formula; see keywords.

To keep the syntax short, the user can use the n function instead of `list()` to specify `s`.

EXAMPLES
Here are some examples for clarity, using an atomic array `x` of 3 dimensions:

```
# extract the first 10 columns:
ss_x(x, 1:10, 2)                # ==> x[ , 1:10, , drop = FALSE]

# extract first 10 indices of every dimension:
ss_x(x, 1:10)                   # ==> x[1:10, 1:10, 1:10, drop = FALSE]

# remove first 10 indices of every dimension:
ss_x(x, 1:10, -Inf)             # ==> x[-1:-10, -1:-10, -1:-10, drop = FALSE]

# extract first 10 rows, all columns, and first 5 layers:
ss_x(x, n(1:10, 0L, 1:5))       # ==> x[1:10, , 1:5, drop = FALSE]

extract the first 10 rows, all columns, and the first 5 layers
ss_x(x, 1:10, c(1, 3))          # ==> x[1:10, , 1:10, drop = FALSE]

# extract first 10 rows, extract all columns, and **remove** first 5 layers:
ss_x(x, n(1:10, 1:5), c(1, -3))  # ==> x[1:10, , -1:-5, drop = FALSE]
```

For a brief explanation on the relationship between flat indices (`i`) and subscripts (`s`, `use`) in arrays, see ss2ii.

Arguments set `row, col, use`

class: atomic matrix
class: recursive matrix
class: data.frame-like

Specifies rows and columns in an atomic matrix, a recursive matrix, or data.frame. The argument `row` and `col` can each be any of the following:

- `NULL` or `0L`, which corresponds to a missing index argument.
- a vector of length 0, in which case no indices are selected for the operation (i.e. empty selection).
- a numeric vector of **strictly positive whole numbers** with dimension indices to select for the operation.
- a **logical** vector of the same length as the corresponding dimension size, giving the dimension indices to select for the operation.
- a **character** vector of index names.
  If a dimension has multiple indices with the given name, ALL the corresponding indices will be selected for the operation.
- a formula; see keywords.

For data.frames, `col` can also be a function.

`use` is set to `1:2` by default.
If `use` is `c(-1, 2)` or `-1`, the row indices will be inverted (i.e. select all rows **except** those specified in `row`).
If `use` is `c(1, -2)` or `-2`, the column indices will be inverted (i.e. select all columns **except** those specified in `col`).
If `use` is `-1:-2`, both the row- and column- indices will be inverted.
The order of `use` is **irrelevant**:
`c(-1, 2)` is the same as `c(2, -1)`, and will not change the evaluation of the `row` and `col` arguments.

Examples:

```
# Extracting rows or columns:
tt_x(x, col = c) # ==> x[, c]
tt_x(x, row = r) # ==> x[r, ]

# Extracting both:
tt_x(x, r, c)           # ==> x[r, c]
tt_x(x, r, c, 2:1)      # ==> x[r, c] (same as previous)

# Excluding rows:
tt_x(x, r, c, c(-1, 2)) # ==> x[-r, c]
tt_x(x, r, c, c(2, -1)) # ==> x[-r, c] (same as previous)
tt_x(x, r, c, -1)       # ==> x[-r, c] (same as previous)

# Excluding columns:
tt_x(x, r, c, c(1, -2)) # ==> x[r, -c]
```

```
tt_x(x, r, c, c(-2, 1)) # ==> x[r, -c] (same as previous)
tt_x(x, r, c, -2)       # ==> x[r, -c] (same as previous)

# Excluding both:
tt_x(x, r, c, -1:-2)    # ==> x[-r, -c]
tt_x(x, r, c, -2:-1)    # ==> x[-r, -c] (same as previous)
```

Argument Pair `slice, use`

class: atomic array
class: recursive array
class: data.frame-like

Relevant only for the `_icom` methods.
The absolute value of `use` specifies the dimension on which argument `slice` is used.
The sign of `use`, just like in the previous argument sets, specifies whether to invert the indices or not.
I.e. if `use = 1` , `slice` specifies which rows to select;
if `use = -2`, `slice` specifies which columns to **not** select;
etc.

The `slice` argument can be any of the following:

- `NULL` or `0L`, which corresponds to a missing index argument.
- a numeric vector of **strictly positive whole numbers** with dimension indices to select for the operation.
- a **logical** vector of the same length as the corresponding dimension size, giving the dimension indices to select for the operation.
- a **character** vector of index names.
  If a dimension has multiple indices with the given name, ALL the corresponding indices will be selected for the operation.
- a formula; see keywords.

One could also give a vector of length `0` for `slice`;
Argument `slice` is only used in the `_icom` methods, and the results are meant to be used inside the regular `[` and `[<-` operators.
Thus the effect of a zero-length index specification depends on the rule-set of `[.class(x)` and `[<-.class(x)`.

All Missing Indices

`NULL` and `0L` in the indexing arguments correspond to a missing argument.
For `s, use`, specifying `use` of length 0 also corresponds to all subscripts being missing.
Thus, for the `_x` methods, using missing indexing arguments for all indexing arguments corresponds to something like the following:

```
x[]
```

Similarly, for the _mod and _set methods, using missing or NULL indexing arguments corresponds to something like the following:

```
x[] <- rp # for replacement
x[] <- tf(x) # for transformation
```

The above is true even when using negative values for use.

Drop

Sub-setting with the generic methods from the 'squarebrackets' R-package using dimensional arguments (s, row, col, use) always use drop = FALSE.
To drop potentially redundant (i.e. single level) dimensions, use the drop function, like so:

```
ss_x(x, s, use) |> drop() # ==> x[..., drop = TRUE]
```

References

Plate T, Heiberger R (2016). *abind: Combine Multidimensional Arrays*. R package version 1.4-5, https://CRAN.R-project.org/package=abind.

---

aaa05_squarebrackets_modify
Regarding Modification

Description

This help page describes the main modification semantics available in 'squarebrackets'.

Base R's default modification

For most average users, R's default copy-on-modify semantics are fine.
The benefits of the indexing arguments from 'squarebrackets' can be combined the [<- operator, through the _icom methods.
The result of the _icom methods can be used inside the regular square-brackets operators.
For example like so:

```
x <- array(...)
my_indices <- ss_icom(x, s, d)
x[my_indices] <- value

y <- data.frame(...)
rows <- tt_icom(y, 1:10, 1, inv = TRUE)
cols <- tt_icom(y, c("a", "b"), 2)
y[rows, cols] <- value
```

thus allowing the user to benefit from the convenient index translations from 'squarebrackets', whilst still using R's default copy-on-modification semantics (instead of the semantics provided by 'squarebrackets').

Explicit Copy

'squarebrackets' provides the `_mod` methods to return a modified copy.
For atomic objects, the copy is a deep copy (the entire object is copied).
This method returns the modified object.
For recursive objects, the copy is a shallow copy:
the `_mod` methods returns the original object, where only the modified subsets are copied, thus preventing unnecessary usage of memory.

Pass-by-Reference

'squarebrackets' provides the `_set` methods to modify by reference, meaning no copy is made at all.
Pass-by-Reference is fastest and the most memory efficient.
But it is also more involved than the other modification forms, and requires more thought.
See squarebrackets_PassByReference for more information.

Arguments `rp` and `tf` for Atomic Objects

Argument `rp`

The `rp` argument is used to replace the values at the specified indices with the values specified in `rp`.
Using the `rp` argument in the modification methods, corresponds to something like the following:

```
x[...] <- rp
```

Argument `tf`

The `tf` argument is used to transform the values at the specified indices through transformation function `tf`. Using the `tf` argument corresponds to something like the following:

```
x[...] <- tf(x[...])
```

where `tf` is a function that **returns** an object of appropriate type and size (so `tf` should not be a pass-by-reference function).

Arguments `rp` and `tf` for Recursive Objects

The `rp` and `tf` arguments work mostly in the same way for recursive objects.
But there are some slight differences.

Argument `rp`
'squarebrackets' demands that `rp` is always provided as a list in the S3 methods for recursive vectors, matrices, and arrays (i.e. lists).
This is to prevent ambiguity with respect to how the replacement is recycled or distributed over the specified indices
(See `Footnote 1` below).

Argument `tf`
Most functions in (base) 'R' are vectorized for atomic objects, but not for lists
(see `Footnote 2` below).
'squarebrackets' will therefore apply transformation function `tf` via `lapply`, like so:

```
x[...] <- lapply(x[...], tf)
```

Arguments `rp` and `tf` for data.frame-like Objects

Replacement and transformations in data.frame-like objects are a bit more flexible than in Lists.

Argument `rp`
`rp` is not always demanded to be a list for data.frame-like objects, only when appropriate (for example, when replacing multiple columns, or when the column itself is a list.)

Argument `tf`
Every column in a data.frame is like an element in a list.
Therefore, when transforming parts of a data.frame with the `tf` argument, lapply‘ is used for transformations across multiple columns.

Recycling and Coercion

Recycling is not allowed in the modification methods.
So, for example, `length(rp)` must be equal to the length of the selected subset, or equal to `1`.

When using Pass-by-Reference semantics, the user should be extra mindful of the auto-coercion rules.
See squarebrackets_coercion for details.

Footnotes

Footnote 1
Consider the following replacement in base 'R':

```
x <-list(1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
x[1:2] <- 2:1
```

What will happen?
Will the `x[1]` be `list(1:2)` and `x[2]` also be `list(1:2)`?
Or will `x[1]` be `list(2)` and `x[2]` be `list(1)`?
It turns out the latter will happen; but this is somewhat ambiguous from the code.
To prevent such ambiguity in your code, 'squarebrackets' demands that `rp` is always provided as a list.

Footnote 2
Most functions in (base) 'R' are vectorized for atomic objects, but not for lists.
One of the reasons is the following:
In an atomic vector `x` of some type `t`, every single element of `x` is a scalar of type `t`.
However, every element of some list `x` can be virtually anything:
an atomic object, another list, an unevaluated expression, even dark magic like `quote(expr =)`.
It is difficult to make a vectorized function for an object with so many unknowns.
Therefore, in the vast majority of the cases, one needs to loop through the list elements.

`aaa06_squarebrackets_options`
squarebrackets Options

Description

This help page explains the various global options that can be set for the 'squarebrackets' package, and how it affects the functionality.

Check Duplicates

argument: chkdup
option: squarebrackets.chkdup
The `_x` methods are the only methods where providing duplicate indices actually make sense.
For the other methods, it doesn't make sense.
Giving duplicate indices usually won't break anything; however, when replacing/transforming or removing subsets, it is almost certainly not the intention to provide duplicate indices.
Providing duplicate indices anyway might lead to unexpected results.
Therefore, for the methods where giving duplicate indices does not make sense, the `chkdup` argument is present.
This argument controls whether the method in question checks for duplicates (`TRUE`) or not (`FALSE`).

Setting `chkdup = TRUE` means the method in question will check for duplicate indices, and give an error when it finds them.

Setting `chkdup = FALSE` will disable these checks, which saves time and computation power, and is thus more efficient.

Since checking for duplicates can be expensive, it is set to FALSE by default.
The default can be changed in the squarebrackets.chkdup option.

Sticky

argument: sticky
option: squarebrackets.sticky
The long_x methods can already handle names (through the use.names argument), as well as attributes specific to the mutatomic class.

Attributes which are not names, and not specific to mutatomic class - henceforth referred to as "other attributes" - are treated differently.
How the long_x methods handle these "other" attributes, is determined by the sticky option and argument.

When sticky = FALSE, the long_x methods will drop all **other** attributes.

By setting sticky = TRUE, all these **other** attributes, except comment and tsp, will be preserved;
The key advantage for this, is that classes that use static attributes (i.e. classes that use attributes that do not change when sub-setting), are automatically supported if sticky = TRUE, and no separate methods have to written for the long_x methods.
Attributes specific to classes like difftime, broadcaster, and more, use static attributes.

Instead of setting sticky = TRUE or sticky = FALSE, one can also specify all classes that use static attributes that you'll be using in the current R session.
In fact, when 'squarebrackets' is **loaded**, the squarebrackets.sticky option is set as follows:

```
squarebrackets.sticky = c(
   "difftime", "broadcaster"
)
```

So in the above default setting, sticky = TRUE for
"difftime", "broadcaster".
Also in the above default setting, sticky = FALSE for other classes.

The reason the long_x methods need the sticky option, is because of the following.
Unlike the ii_, ii_, ss_, and ss_ methods, the long_x methods are not wrappers around the [and [<- operators.
Therefore, most [- S3 methods for highly specialized classes are not readily available for the long_x methods.
Which in turn means important class-specific attributes are not automatically preserved.
The sticky option is a convenient way to support a large number of classes, without having to write specific methods for them.

For specialized classes that use attributes that **do** change when sub-setting, separate dispatches for the long_x methods need to be written.
Package authors are welcome to create method dispatches for their own classes for these methods.

`aaa07_squarebrackets_ellipsis`
Ellipses Usage in 'squarebrackets'

Description

Due to how the S3 method dispatch system works in 'R', all generic methods have the ellipsis argument (`...`).
For the user's safety, 'squarebrackets' does check that the user doesn't accidentally add arguments that make no sense for that method.

`aaa08_squarebrackets_stride`
Argument stride

Description

'squarebrackets' provides the long_x and long_set methods to perform sub-set operations on the interior of a vector, without any indexing vector at all.
The advantage of not using an indexing vector for sub-set operations on a long vector, is that it may safe the user a large amount of memory. Instead of an indexing vector, they use the `stride` argument.
The following can be used in the `stride` argument:

- a stride_v object:
  Use this `stride` type to specify value-based subsets, like `y == v`, where `y` is a vector of the same length as `x`, and `v` is a value (or range of values) `y` might contain.

- a stride_seq object:
  Use this `stride` type to specify a sequence in the form of `seq(from, to, by)`, without actually allocating a sequence indexing vector.
- a stride_ptrn object:
  Use this `stride` type to specify a patterned sequence in the form of `(start:end)[pattern]`,
  where `start` and `end` are natural scalars and `pattern` is a logical vector.
  stride_ptrn specifies this sequence without actually allocating an indexing vector.
- A formula in the form of `~ from:to:by:use`, where `from`, `to` and `by` are natural scalars, and `use` is `1` or `-1`.
  This will be interpreted as `stride_seq(from, to, by, use)`.

- A formula in the form of `~ start:end:ptrn:use`, where `start` and `end` are natural scalars, `use` is `1` or `-1`, and `ptrn` is a logical vector.
  This will be interpreted as `stride_ptrn(start, end, ptrn, use)`.

In both formula forms, the `.N` keyword is available, which equals `length(x)` (where `x` is the input vector).
For example,
`~ 2:(.N - 1):c(TRUE, FALSE, FALSE, TRUE):1`
is equivalent to `~ 2:(length(x) - 1):c(TRUE, FALSE, FALSE, TRUE):1`,
and will be interpreted as `stride_ptrn(2, length(x) - 1, c(TRUE, FALSE, FALSE, TRUE), 1)`.

Examples

```
# extract all elements of x with the name "a":
nms <- c(letters, LETTERS, month.abb, month.name) |> rep_len(1e6)
x <- mutatomic(1:1e6, names = nms)
head(x)
stride <-  stride_v(names(x), v = "a")
long_x(x, stride) |> head()


# find all x smaller than or equal to 5, and replace with `-1000`:
stride <- stride_v(x, v = c(-Inf, 5))
long_set(x, stride, rp = -1000L)
head(x, n = 10)




x <- mutatomic(1:1e7)

# extract elements 2 to 9
long_x(x, ~ 2:9:1:1)

# reverse:
long_x(x, ~ 9:2:1:1)

# remove all elements except the last 10:
long_x(x, ~ 1:(.N - 10):1:-1)

# replace every other element:
x <- mutatomic(1:1e7)
long_set(x, ~ 2:.N:2:1, rp = -1)
head(x)

# replace all elements except the first element:
x <- mutatomic(1:1e7)
long_set(x, ~1:1:1:-1, rp = -1)
head(x)


# extract pattern c(TRUE, FALSE, FALSE, TRUE) from first 20 elements:
```

```
x <- mutatomic(1:1e7)
long_x(x, ~ 1:20:c(TRUE, FALSE, FALSE, TRUE):1)
```

`aaa09_squarebrackets_PassByReference`
Regarding Modification By Reference

Description

This help page describes how modification using "pass-by-reference" semantics is handled by the 'squarebrackets' package.

"Pass-by-reference" refers to modifying a mutable object, or a subset of a mutable object, without making any copies at all.

This help page does not explain all the basics of pass-by-reference semantics, as this is treated as prior knowledge.
All functions/methods in the 'squarebrackets' package with the word "set" in the name use pass-by-reference semantics.

Advantages and Disadvantages

The main advantage of pass-by-reference is that much less memory is required to modify objects, and modification is also generally faster.
But it does have several disadvantages.

First, the coercion rules are slightly different: see squarebrackets_coercion.

Second, if 2 or more variables refer to exactly the same object (i.e. have the same address), changing one variable also changes the other ones.
I.e. the following code,

```
x <- y <- mutatomic(1:16)
ii_set(x, 1:6, rp = 8)
```

modifies not just `x`, but also `y`.
This is true even if one of the variables is locked (see bindingIsLocked).
I.e. the following code,

```
x <- mutatomic(1:16)
y <- x
lockBinding("y", environment())
ii_set(x, i = 1:6, rp = 8)
```

modifies both `x` and `y` without error, even though `y` is a locked constant.

Mutable vs Immutable Classes

With the exception of environments, most of base R's S3 classes are treated as immutable:
Modifying an object in 'R' will make a copy of the object, something called 'copy-on-modify' semantics.

A prominent mutable S3 class is the `data.table` class, which is a mutable data.frame class, and supported by 'squarebrackets'.
Similarly, 'squarebrackets' adds a class for mutable atomic objects:
mutatomic.

Material vs Immaterial objects

Most objects in 'R' are material objects:
the values an object contains are actually stored in memory.
For example, given `x <- rnorm(1e6)`, `x` is a material object:
1 million values (decimal numbers, in this case) are actually stored in memory.

In contrast, ActiveBindings are immaterial:
They are objects that, when accessed, call a function to generate values on the fly, rather than actually storing values.

Since immaterial objects do not actually store the values in memory, the values obviously also cannot be changed in memory.
Therefore, Pass-by-Reference semantics don't work on immaterial objects.

ALTREP

The mutatomic constructors (i.e. mutatomic, as.mutatomic, etc.) will automatically materialize ALTREP objects, to ensure consistent behaviour for 'pass-by-reference' semantics.

A `data.table` can have ALTREP columns.
A `data.tables` will coerce the column to a materialized column when it is modified, even by reference.

Input Variable

Methods/functions that perform in-place modification by reference only works on objects that actually exist as an actual variable, similar to functions in the style of `some_function(x, ...) <- value`.
Thus things like any of the following,
`ii_set(1:10, ...)`, `ii_set(x$a, ...)`, or `ii_set(base::letters)`,
will not and should not work.

Lock Binding

Mutable classes are, as the name suggests, meant to be mutable.
Locking the binding of a mutable object is fruitless.
To ensure an object cannot be modified by any of the methods/functions from 'squarebrackets', 2 things must be true:

- the object must be an immutable class.
- the binding must be **locked** (see lockBinding).

Protection

Due to the properties described above in this help page, 'squarebrackets' protects the user from do something like the following:

```
# letters = base::letters
ii_set(letters, i = 1, rp = "XXX")
```

'squarebrackets' will give an error when running the code above, because:

1. most addresses in `baseenv()` are protected;
2. immutable objects are disallowed (you'll have to create a mutable object, which will create a copy of the original, thus keeping the original object safe from modification by reference);
3. locked bindings are disallowed.

Examples

```
# the following code demonstrates how locked bindings,
# such as `base::letters`,
# are being safe-guarded

x <- list(a = base::letters)
myref <- x$a # view of a list
address(myref) == address(base::letters) # TRUE: point to the same memory
bindingIsLocked("letters", baseenv()) # base::letters is locked ...
bindingIsLocked("myref", environment()) # ... but this pointer is not!

if(requireNamespace("tinytest")) {
  tinytest::expect_error(
    ii_set(myref, i = 1, rp = "XXX") # this still gives an error though ...
  )
}

is.mutatomic(myref) # ... because it's not of class `mutatomic`


x <- list(
  a = as.mutatomic(base::letters) # `as.mutatomic()` makes a copy
)
myref <- x$a # view of a list
address(myref) == address(base::letters) # FALSE: it's a copy
```

```
ii_set(
  myref, i = 1, rp = "XXX"  # modifies x, does NOT modify `base::letters`
)
print(x) # x is modified
base::letters # but this still the same
```

`aaa10_squarebrackets_coercion`

Auto-Coercion Rules for Mutable Objects

Description

This help page describes the auto-coercion rules of the mutable classes, as they are handled by the 'squarebrackets' package.
This useful information for users who wish to intend to employ Pass-by-Reference semantics as provided by 'squarebrackets'.

mutatomic

coercion_through_copy: YES
coercion_by_reference: NO
Mutable atomic objects are automatically coerced to fit the modified subset values, when modifying through copy, just like regular atomic classes.
For example, replacing one or multiple values in an integer vector (type `int`) with a decimal number (type `dbl`) will coerce the entire vector to type `dbl`.

Replacing or transforming subsets of mutable atomic objects **by reference** does not support coercion. Thus, for example, the following code,

```
x <- mutatomic(1:16)
ii_set(x, i = 1:6, rp = 8.5)
#> coercing replacement to integer
print(x)
#>  [1]  8  8  8  8  8  8  7  8  9 10 11 12 13 14 15 16
#> mutatomic
#> typeof:  integer
```

gives `c(rep(8, 6) 7:16)` instead of `c(rep(8.5, 6), 7:16)`, because `x` is of type `integer`, so `rp` is interpreted as type `integer` also.

data.table, when replacing/transforming whole columns

coercion_through_copy: YES
coercion_by_reference: YES
A data.table is actually a list made mutable, where each column is itself a list. As such, replacing/transforming whole columns using `data.table::set()`, without specifying rows (not even `i = 1:nrow(x)`), allows completely changing the type of the column.

data.table, when partially replacing/transforming columns

coercion_through_copy: YES
coercion_by_reference: NO
If rows are specified in the `data.table::set()` function (and functions that internally use `data.table::set()`), and thus not all values of columns but parts(i.e. rows) of columns are replaced, no auto-coercion takes place.
I.e.: replacing/transforming a value in an integer (`int`) column to become `1.5`, will not coerce the column to the decimal type (`dbl`); instead, the replacement value `1.5` is coerced to integer `1`.

Using R's native copy-on-modify semantics (for example by changing a `data.table` into a `data.frame`) allows for coercion even when partially replacing/transforming columns.

Views of Lists

Regular lists are treated as immutable by 'squarebrackets'.
But remember that a list is a (potentially hierarchical) structure of references to other objects.
Thus, even if a list itself is not treated as mutable, subsets of a list which are themselves mutable classes, are mutable.
For example, if you have a list of mutatomic objects, the mutatomic objects themselves are mutable.
Therefore, the following will work:

```
x <- list(
 a = mutatomic(letters[1:10]),
 b = mutatomic(letters[11:20])
)
myref <- x$a
ii_set(myref, 1, rp = "xxx")
```

Notice in the above code that `myref` is not a copy of `x$a`, since they have the same address.
Thus changing `myref` also changes `x$a`.
In other words: `myref` is what could be called a "view" of `x$a`.
Notice also that ii_set`(x$a, ...)` will not work.
This is because stopifnot_ma_safe2mutate will give an error if `x` is not an **actual variable**, similar to in-place functions in the style of `` `myfun()<-` ``.

The auto-coercion rules of Views of Lists, depends entirely on the object itself.

Thus if the View is a data.table, coercion rules of data.tables apply.
And if the View is a mutatomic object, coercion rules of mutatomic objects apply, etc.

Examples

```
# Coercion examples - mutatomic ====

x <- as.mutatomic(1:16)
ii_set(x, i = 1:6, rp = 8.5) # 8.5 coerced to 8, because `x` is of type `integer`
print(x)

################################################################################

# Coercion examples - data.table - whole columns ====

# tt_mod():
obj <- data.table::data.table(
  a = 1:10, b = letters[1:10], c = 11:20, d = factor(letters[1:10])
)
str(obj) # notice that columns "a" and "c" are INTEGER (`int`)
tt_mod(
  obj, col = is.numeric,
  tf = sqrt # SAFE: obs = NULL, so coercion performed
)

# tt_set():
tt_set(
  obj, col = is.numeric,
  tf = sqrt # SAFE: obs = NULL, so coercion performed
)
str(obj)

################################################################################


# Coercion examples - data.table - partial columns ====

# tt_mod():
obj <- data.table::data.table(
  a = 1:10, b = letters[1:10], c = 11:20, d = factor(letters[1:10])
)
str(obj) # notice that columns "a" and "c" are INTEGER (`int`)

tt_mod(
  obj, with(obj,  (a >= 2) & (c <= 17)), is.numeric,
  tf = sqrt # SAFE: coercion performed
)

# tt_set():
obj <- data.table::data.table(
  a = 1:10, b = letters[1:10], c = 11:20, d = factor(letters[1:10])
)
str(obj) # notice that columns "a" and "c" are INTEGER (`int`)
tt_set(
  obj, with(obj,  (a >= 2) & (c <= 17)), is.numeric,
```

```
  tf = sqrt
  # WARNING: sqrt() results in `dbl`, but columns are `int`, so decimals lost
)
print(obj)

obj <- data.table::data.table(
  a = 1:10, b = letters[1:10], c = 11:20, d = factor(letters[1:10])
)
str(obj)
tt_set(obj, col = is.numeric, tf = as.numeric) # first coerce type by whole columns
str(obj)
tt_set(
  obj,
  with(obj,  (a >= 2) & (c <= 17)), is.numeric,
  tf = sqrt # SAFE: coercion performed by dt_setcoe(); so no warnings
)
print(obj)


################################################################################

# View of List ====

x <- list(
 a = data.table::data.table(cola = 1:10, colb = letters[1:10]),
 b = data.table::data.table(cola = 11:20, colb = letters[11:20])
)
print(x)
myref <- x$a
address(myref) == address(x$a) # they are the same
tt_set(myref, col = "cola", tf = \(x)x^2)
print(x) # notice x has been changed
```

`aaa11_squarebrackets_keywords`
Advanced Index Specification with Keywords

Description

Instead of the usual indexing types, formulas can be used to specify indices in vectors, arrays, or data.frames that can evaluate keywords.

The following keywords are available:

- `.M`: the given margin/dimension; `0` if not relevant.
- `.Nms`: the (dim)names at the given margin.
- `.N`: the size of a given dimension (if `.M` is not `0`) or else the length of `x`.
- `.I`: equal to `seq_len(.N)`.
- `.bi(...)`: a **function** to specify bilateral indices. See the `Details` section below.
- `.x`: the input variable `x` itself.

These keywords can be used inside formulas to specify more advanced indices.

Here are a few examples of advanced indexing in arrays:

```
x <- matrix(1:20, 5, 4)
dimnames(x) <- list(month.abb[1:5], month.abb[1:4])
print(x)
#>     Jan Feb Mar Apr
#> Jan   1   6  11  16
#> Feb   2   7  12  17
#> Mar   3   8  13  18
#> Apr   4   9  14  19
#> May   5  10  15  20


# select first half of rows, select first & last column:
ss_x(x, n(~ 1:round(.N/2), ~ .bi(1, -1)))
#>     Jan Apr
#> Jan   1  16
#> Feb   2  17


# extract where rownames contain "r" & where colnames DO NOT contain "r":
ss_x(x, ~ grep("r", .Nms), use = c(1, -2))
#>     Jan Feb
#> Mar   3   8
#> Apr   4   9


# reverse order of all dimensions:
ss_x(x, ~ .bi(-.I))
#>     Apr Mar Feb Jan
#> May  20  15  10   5
#> Apr  19  14   9   4
#> Mar  18  13   8   3
#> Feb  17  12   7   2
#> Jan  16  11   6   1
```

Details

Bilateral Indices

The `.bi(...)` function is used to specify bilateral indices.
One can specify both positive and negative numbers.
Positive numbers (for example `~ .bi(1:10)`) specify indices work like normally.
Negative numbers specify indices from the end;
i.e. `~ .bi(-2)` would be equivalent to `length(x) - 1` if using an `ii_` method, or `dim(x)[.M] - 1` if using a `ss_` or `tt_` method.

The usage of `.bi(...)` is similar to the `c(...)` function, in that multiple vectors can be concatenated to one.
And one can specify both negative and positive numbers together.
For example: `~ .bi(-10:-1, 10:1)`.

One can also use the other keywords inside `.bi()`.
For example, `~ .bi(-.I)` can be used to reverse a vector, or reverse a dimension of an array or data.frame.

`developer_ci` *Construct Indices*

Description

These functions construct indices.

- `ci_ii()` constructs an integer vector flat/interior indices.
- `ci_margin()` constructs an integer vector of indices for one particular dimension margin.
- `ci_ss()` constructs a list of integer subscripts.

Usage

```
ci_ii(
  x,
  i = NULL,
  use = 1L,
  chkdup = FALSE,
  uniquely_named = FALSE,
  .abortcall = sys.call()
)

ci_margin(
  x,
  slice = NULL,
  margin,
  use = 1L,
  chkdup = FALSE,
  uniquely_named = FALSE,
  .abortcall = sys.call()
)

ci_ss(
  x,
  s = NULL,
  use = 1:ndim(x),
  chkdup = FALSE,
  uniquely_named = FALSE,
  .abortcall = sys.call()
)
```

Arguments

x	the object for which the indices are meant.
i, s, use, slice, margin	See squarebrackets_index_args.
chkdup	see squarebrackets_options. for performance: set to FALSE
uniquely_named	Boolean, indicating if the user knows a-priori that the relevant names of x are unique. If set to TRUE, speed may increase. But specifying TRUE when the relevant names are not unique will result in incorrect output.
.abortcall	environment where the error message is passed to.

Value

An integer vector of constructed indices.

Examples

```
x <- matrix(1:25, 5, 5)
colnames(x) <- c("a", "a", "b", "c", "d")
print(x)

bool <- sample(c(TRUE, FALSE), 5, TRUE)
int <- 1:4
chr <- c("a", "a")
tci_bool(bool, nrow(x))
tci_int(int, ncol(x), -1)
tci_chr(chr, colnames(x))

ci_ii(x, 1:10)
ci_margin(x, 1:4, 2)
ci_ss(x, n(~ .bi(-1:-5), 1:4), 1:2)
```

---

developer_tci *Type Cast Indices*

Description

These functions typecast indices to proper integer indices.

Usage

```
tci_atomic(
  indx,
  n,
  nms,
```

```
  use = 1,
  chkdup = FALSE,
  uniquely_named = FALSE,
  .abortcall = sys.call()
)

tci_bool(indx, n, use = 1L, .abortcall = sys.call())

tci_int(indx, n, use = 1L, chkdup = FALSE, .abortcall = sys.call())

tci_chr(
  indx,
  nms,
  use = 1L,
  chkdup = FALSE,
  uniquely_named = FALSE,
  .abortcall = sys.call()
)

tci_formula(x, m, form, .abortcall)
```

Arguments

indx	the indices to typecast
n	the relevant size, when typecasting integer or logical indices. Examples: • If the target is row indices, input nrow for n. • If the target is flat indices, input the length for n.
nms	the relevant names, when typecasting character indices. Examples: • If the target is row indices, input row names for nms. • If the target is flat indices, input flat names for nms.
use	1 or -1, indicating how to use the indices. See squarebrackets_index_args.
chkdup	see squarebrackets_options. for performance: set to FALSE
uniquely_named	Boolean, indicating if the user knows a-priori that the relevant names of x are unique. If set to TRUE, speed may increase. But specifying TRUE when the relevant names are not unique will result in incorrect output.
.abortcall	environment where the error message is passed to.
x	the input variable
m	a number giving the margin; set to 0 for dimensionless vectors.
form	a formula; see keywords.

Value

An integer vector of type-cast indices.

Examples

```
x <- matrix(1:25, 5, 5)
colnames(x) <- c("a", "a", "b", "c", "d")
print(x)

bool <- sample(c(TRUE, FALSE), 5, TRUE)
int <- 1:4
chr <- c("a", "a")
tci_bool(bool, nrow(x))
tci_int(int, ncol(x), -1)
tci_chr(chr, colnames(x))

ci_ii(x, 1:10)
ci_margin(x, 1:4, 2)
ci_ss(x, n(~ .bi(-1:-5), 1:4), 1:2)
```

---

dropl *Helper Functions for Sub-Set Operations on Recursive Objects*

Description

'squarebrackets' provides some helper functions for sub-set operations on recursive objects (lists and data.frames).

`dropl(x)` returns `x[[1L]]` if `length(x) == 1`, and returns `x` otherwise.
This can be used for the `_x` methods on recursive objects, where one would like to extract the contents of the singular selection.

Usage

```
dropl(x)
```

Arguments

x	a list.

Value

For `dropl()`:
If `length(x) == 1L`, `dropl(x)` returns `x[[1L]]`;
otherwise it returns the original `x` unchanged.

Examples

```
obj <- as.list(1:10)
ii_x(obj, 1) |> dropl() # equivalent to obj[[1L]]
```

idx_by *Compute Grouped Indices*

Description

Given:

- a sub-set function `f`;
- an object `x` with its margin `m`;
- and a grouping factor `grp`;

the `idx_by()` function takes `indices` **per group** `grp`.
The result of `idx_by()` can be supplied to the indexing arguments (see squarebrackets_index_args) to perform **grouped** subset operations.

Usage

```
idx_by(x, m, f, grp, parallel = FALSE, mc.cores = 1L)
```

Arguments

`x`	the object from which to compute the indices.
`m`	a single non-negative integer giving the margin for which to compute indices. For flat indices or for non-dimensional objects, use `m = 0L`.
`f`	a subset function to be applied per group on `indices`. If `m == 0L`, `indices` is here defined as `setNames(1:length(x), names(x))`. If `m > 0L`, `indices` is here defined as `setNames(1:dim(x)[m], dimnames(x)[[m]])`. The function must produce a character or integer vector as output. For example, to subset the last element per group, specify: `f = last`
`grp`	a factor giving the groups.
`parallel`, `mc.cores`	see BY.

Value

A vector of indices.

Examples

```
# vectors ====
(a <- 1:20)
(grp <- factor(rep(letters[1:5], each = 4)))

# get the last element of `a` for each group in `grp`:
s <- list(idx_by(a, 0L, last, grp))
ss_x(cbind(a, grp), s, 1L)
```

```
# data.frame ====
x <- data.frame(
  a = sample(1:20),
  b = letters[1:20],
  group = factor(rep(letters[1:5], each = 4))
)
print(x)
# get the first row for each group in data.frame `x`:
row <- idx_by(x, 1, first, x$group)
tt_x(x, row)
# get the first row for each group for which a > 10:
x2 <- tt_x(x, with(x,  a > 10))
row <- na.omit(idx_by(x2, 1, first, x2$group))
tt_x(x2, row)
```

---

ii_icom *Compute Indices for Copy-On-Modify Substitution*

Description

The _icom() methods compute indices for copy-on-modify substitution.

```
x <- array(...)
my_ss2ii <- ss_icom(x, s, use)
x[my_ss2ii] <- value

y <- data.frame(...)
rows <- tt_icom(y, 1:10, 1, inv = TRUE)
cols <- tt_icom(y, c("a", "b"), 2)
y[rows, cols] <- value
```

thus allowing the user to benefit from the convenient index translations from 'squarebrackets', whilst still using R's default copy-on-modification semantics (instead of the semantics provided by 'squarebrackets').

Usage

```
ii_icom(x, i = NULL, use = 1, ...)

ss_icom(x, s = NULL, use = 1:ndim(x), ...)

tt_icom(x, slice, use, ...)

## Default S3 method:
ii_icom(
```

```
  x,
  i = NULL,
  use = 1,
  ...,
  chkdup = getOption("squarebrackets.chkdup", FALSE)
)

## Default S3 method:
ss_icom(
  x,
  s = NULL,
  use = 1:ndim(x),
  ...,
  chkdup = getOption("squarebrackets.chkdup", FALSE)
)

## Default S3 method:
tt_icom(
  x,
  slice = NULL,
  use = NULL,
  ...,
  chkdup = getOption("squarebrackets.chkdup", FALSE)
)
```

Arguments

x	vector, matrix, array, or data.frame; both atomic and recursive objects are supported.
i, s, slice, use	See squarebrackets_index_args. Duplicates are not allowed.
...	see squarebrackets_ellipsis.
chkdup	see squarebrackets_options. for performance: set to FALSE

Value

A strictly positive numeric vector of indices.

Examples

```
# atomic ====

x <- 1:10
x[ii_icom(x, \(x)x>5)] <- -5
print(x)

x <- array(1:27, dim = c(3,3,3))
x[ss_icom(x, n(1:2, 1:2), c(1,3))] <- -10
print(x)
```

```
################################################################################


# recursive ====

x <- as.list(1:10)
x[ii_icom(x, \(x)x>5)] <- -5
print(x)

x <- array(as.list(1:27), dim = c(3,3,3))
x[ss_icom(x, n(1:2, 1:2), c(1,3))] <- -10
print(x)


x <- data.frame(
  a = sample(c(TRUE, FALSE, NA), 10, TRUE),
  b = 1:10,
  c = rnorm(10),
  d = letters[1:10],
  e = factor(letters[11:20])
)
rows <- tt_icom(x, 1:5, -1)
cols <- tt_icom(x, c("b", "a"), 2)
x[rows, cols] <- NA
print(x)
```

---

ii_mod *Method to Return a Copy of an Object With Modified Subsets*

Description

Methods to return a copy of an object with modified subsets.
For modifying subsets using R's default copy-on-modification semantics, see the _icom methods.

Usage

```
ii_mod(x, i = NULL, use = 1, ..., rp, tf)

ss_mod(x, s = NULL, use = rdim(x), ..., rp, tf)

tt_mod(x, row = NULL, col = NULL, use = 1:2, ..., rp, tf)

## Default S3 method:
ii_mod(
  x,
  i = NULL,
  use = 1,
  ...,
```

```
  rp,
  tf,
  chkdup = getOption("squarebrackets.chkdup", FALSE)
)

## Default S3 method:
ss_mod(
  x,
  s = NULL,
  use = rdim(x),
  ...,
  rp,
  tf,
  chkdup = getOption("squarebrackets.chkdup", FALSE)
)

## Default S3 method:
tt_mod(
  x,
  row = NULL,
  col = NULL,
  use = 1:2,
  ...,
  rp,
  tf,
  chkdup = getOption("squarebrackets.chkdup", FALSE)
)

## S3 method for class 'data.frame'
tt_mod(
  x,
  row = NULL,
  col = NULL,
  use = 1:2,
  ...,
  rp,
  tf,
  chkdup = getOption("squarebrackets.chkdup", FALSE)
)
```

Arguments

x	see squarebrackets_supported_structures.
i, use, s, row, col	See squarebrackets_index_args. An empty index selection returns the original object unchanged.
...	see squarebrackets_ellipsis.
rp, tf	see squarebrackets_modify.
chkdup	see squarebrackets_options. for performance: set to FALSE

Details

Transform or Replace

Specifying argument `tf` will transform the subset.
Specifying `rp` will replace the subset.
One cannot specify both `tf` and `rp`. It's either one set or the other.

Value

A copy of the object with replaced/transformed values.

Examples

```
# atomic objects ====

obj <- matrix(1:16, ncol = 4)
colnames(obj) <- c("a", "b", "c", "a")
print(obj)
rp <- -1:-9
ss_mod(obj, n(1:3), 1:ndim(obj), rp = rp)
# above is equivalent to  obj[1:3, 1:3] <- -1:-9; obj
ii_mod(obj, i = \(x)x<=5, rp = -1:-5)
# above is equivalent to  obj[obj <= 5] <- -1:-5; obj
ss_mod(obj, n("a"), 2L, rp = -1:-8)
# above is equivalent to  obj[, which(colnames(obj) %in% "a")] <- -1:-8; obj
ss_mod(obj, n(1:3), 1:ndim(obj), tf = \(x) -x)
# above is equivalent to  obj[1:3, 1:3] <- (-1 * obj[1:3, 1:3]); obj
ii_mod(obj, i = \(x)x <= 5, tf = \(x) -x)
# above is equivalent to  obj[obj <= 5] <- (-1 * obj[obj <= 5]); obj

obj <- array(1:64, c(4,4,3))
print(obj)
ss_mod(obj, n(1:3, 1:2), c(1,3), rp = -1:-24)
# above is equivalent to obj[1:3, , 1:2] <- -1:-24
ii_mod(obj, i = \(x)x <= 5, rp = -1:-5)
# above is equivalent to obj[obj <= 5] <- -1:-5

################################################################################


# lists ====

obj <- list(a = 1:10, b = letters[1:11], c = 11:20)
print(obj)
ii_mod(obj, "a", rp = list(1L))
# above is equivalent to  obj[["a"]] <- 1L; obj
ii_mod(obj, is.numeric, rp = list(-1:-10, -11:-20))
# above is equivalent to  obj[which(sapply(obj, is.numeric))] <- list(-1:-10, -11:-20); obj

obj <- rbind(
  lapply(1:4, \(x)sample(c(TRUE, FALSE, NA))),
  lapply(1:4, \(x)sample(1:10)),
  lapply(1:4, \(x)rnorm(10)),
```

```
  lapply(1:4, \(x)sample(letters))
)
colnames(obj) <- c("a", "b", "c", "a")
print(obj)
ss_mod(obj, n(1:3), 1:ndim(obj),rp = n(-1))
# above is equivalent to obj[1:3, 1:3] <- list(-1)
ii_mod(obj, i = is.numeric, rp = n(-1))
# above is equivalent to obj[sapply(obj, is.numeric)] <- list(-1)
ss_mod(obj, n("a"), 2L, rp = n(-1))
# above is equivalent to
# obj[, lapply(c("a", "a"), \(i) which(colnames(obj) == i)) |> unlist()] <- list(-1)


obj <- array(as.list(1:64), c(4,4,3))
print(obj)
ss_mod(obj, n(1:3, 1:2), c(1,3), rp = as.list(-1:-24))
# above is equivalent to obj[1:3, , 1:2] <- as.list(-1:-24)
ii_mod(obj, i = \(x) x <= 5, rp = as.list(-1:-5))
# above is equivalent to obj[sapply(onj, \(x) x <= 5)] <- as.list(-1:-5)


################################################################################

# data.frame-like objects  - whole columns ====

obj <- data.frame(a = 1:10, b = letters[1:10], c = 11:20, d = factor(letters[1:10]))
str(obj) # notice that columns "a" and "c" are INTEGER (`int`)
tt_mod(
  obj, col = is.numeric,
  tf = sqrt
)

################################################################################

# data.frame-like objects  - partial columns ====

obj <- data.frame(a = 1:10, b = letters[1:10], c = 11:20, d = factor(letters[1:10]))
str(obj) # notice that columns "a" and "c" are INTEGER (`int`)

tt_mod(
  obj, with(obj,  (a > 2) & (c < 17)), is.numeric,
  tf = sqrt
)
tt_mod(
  obj, with(obj,  (a > 2) & (c < 17)), is.numeric,
  tf = sqrt
)
tt_mod(
  obj, with(obj,  (a > 2) & (c < 17)), is.numeric,
  tf = sqrt
)
```

ii_set *Methods to Modify Subsets of a Mutable Object By Reference*

Description

Methods to replace or transform a subset of a supported mutable object using pass-by-reference semantics.

Usage

```
ii_set(x, i = NULL, use = 1, ..., rp, tf)

ss_set(x, s = NULL, use = rdim(x), ..., rp, tf)

tt_set(x, row = NULL, col = NULL, use = 1:2, ..., rp, tf)

## Default S3 method:
ii_set(
  x,
  i = NULL,
  use = 1,
  ...,
  rp,
  tf,
  chkdup = getOption("squarebrackets.chkdup", FALSE)
)

## Default S3 method:
ss_set(
  x,
  s = NULL,
  use = rdim(x),
  ...,
  rp,
  tf,
  chkdup = getOption("squarebrackets.chkdup", FALSE)
)

## Default S3 method:
tt_set(
  x,
  row = NULL,
  col = NULL,
  use = 1:2,
  ...,
  rp,
  tf,
  chkdup = getOption("squarebrackets.chkdup", FALSE)
)
```

```
## S3 method for class 'data.table'
tt_set(
  x,
  row = NULL,
  col = NULL,
  use = 1:2,
  ...,
  rp,
  tf,
  chkdup = getOption("squarebrackets.chkdup", FALSE)
)
```

Arguments

x	a **variable** belonging to one of the supported mutable classes.
i, use, s, row, col	See squarebrackets_index_args. An empty index selection leaves the original object unchanged.
...	see squarebrackets_ellipsis.
rp, tf	see squarebrackets_modify.
chkdup	see squarebrackets_options. for performance: set to FALSE

Details

Transform or Replace
Specifying argument `tf` will transform the subset. Specifying `rp` will replace the subset. One cannot specify both `tf` and `rp`. It's either one set or the other.

Value

Returns: VOID. This method modifies the object by reference.
Do not use assignments like `x <- ii_set(x, ...)`.
Since this function returns void, you'll just get `NULL`.

Examples

```
# mutatomic objects ====

gen_mat <- function() {
  obj <- as.mutatomic(matrix(1:16, ncol = 4))
  colnames(obj) <- c("a", "b", "c", "a")
  return(obj)
}
```

```
obj <- obj2 <- gen_mat()
print(obj)

ss_set(obj, n(1:3), 1:ndim(obj), rp = -1:-9)
print(obj2)
# above is like x[1:3, 1:3] <- -1:-9, but using pass-by-reference

obj <- obj2 <- gen_mat()
obj

ii_set(obj, i = \(x) x <= 5, rp = -1:-5)
print(obj2)
# above is like x[x <= 5] <- -1:-5, but using pass-by-reference

obj <- obj2 <- gen_mat()
obj

ss_set(obj, n("a"), 2L, rp = cbind(-1:-4, -5:-8))
print(obj2)
# above is like x[, "a"] <- cbind(-1:-4, -5:-8), but using pass-by-reference

obj <- obj2 <- gen_mat()
obj

ss_set(obj, n(1:3), 1:ndim(obj), tf = \(x) -x)
print(obj2)
# above is like x[1:3, 1:3] <- -1 * x[1:3, 1:3], but using pass-by-reference

obj <- obj2 <- gen_mat()
obj

ii_set(obj, i = \(x) x <= 5, tf = \(x) -x)
print(obj2)
# above is like x[x <= 5] <- -1 * x[x <= 5], but using pass-by-reference

obj <- obj2 <- gen_mat()
obj

ss_set(obj, n("a"), 2L, tf = \(x) -x)
obj2
# above is like x[, "a"] <- -1 * x[, "a"], but using pass-by-reference


gen_array <- function() {
  as.mutatomic(array(1:64, c(4,4,3)))
}
obj <- obj2 <- gen_array()
obj

ss_set(obj, n(1:3, 1:2, c(1, 3)), 1:3, rp = -1:-12)
print(obj2)
# above is like x[1:3, , 1:2] <- -1:-12, but using pass-by-reference


obj <- obj2 <- gen_array()
obj
ii_set(obj, i = \(x)x <= 5, rp = -1:-5)
```

```
print(obj2)
# above is like x[x <= 5] <- -1:-5, but using pass-by-reference



################################################################################

# data.table ====

obj <- data.table::data.table(a = 1:10, b = letters[1:10], c = 11:20, d = factor(letters[1:10]))
str(obj) # notice that columns "a" and "c" are INTEGER (`int`)
tt_set(
  obj, with(obj,  (a >= 2) & (c <= 17)), is.numeric,
  tf = sqrt # WARNING: sqrt() results in `dbl`, but columns are `int`, so decimals lost
)
print(obj)

obj <- data.table::data.table(a = 1:10, b = letters[1:10], c = 11:20, d = factor(letters[1:10]))
tt_set(obj, col = is.numeric, tf = as.numeric) # first coerce type by whole columns
str(obj)
tt_set(obj, with(obj,  (a >= 2) & (c <= 17)), is.numeric,
  tf = sqrt # SAFE: coercion performed by column first
)
print(obj)

obj <- data.table::data.table(a = 1:10, b = letters[1:10], c = 11:20, d = factor(letters[1:10]))
str(obj) # notice that columns "a" and "c" are INTEGER (`int`)
tt_set(obj,
        col = is.numeric,
        tf = sqrt # SAFE: obs = NULL, so coercion performed
)
str(obj)
```

`ii_x` *Methods to Extract, Exchange, Exclude, or Duplicate Subsets of an Object*

Description

Methods to extract, exchange, exclude, or duplicate (i.e. repeat x times) subsets of an object.

Usage

```
ii_x(x, i = NULL, use = 1, ...)

ss_x(x, s = NULL, use = 1:ndim(x), ...)

tt_x(x, row = NULL, col = NULL, use = 1:2, ...)

## Default S3 method:
ii_x(x, i = NULL, use = 1, ...)
```

```
## Default S3 method:
ss_x(x, s = NULL, use = 1:ndim(x), ...)

## Default S3 method:
tt_x(x, row = NULL, col = NULL, use = 1:2, ...)

## S3 method for class 'data.frame'
tt_x(x, row = NULL, col = NULL, use = 1:2, ...)
```

Arguments

x	see squarebrackets_supported_structures.
i, use, s, row, col	See squarebrackets_index_args. Duplicates are allowed, resulting in duplicated indices. An empty index selection results in an empty object of length 0.
...	see squarebrackets_ellipsis.

Value

Returns a copy of the sub-setted object.

Examples

```
# atomic objects ====

obj <- matrix(1:16, ncol = 4)
colnames(obj) <- c("a", "b", "c", "a")
print(obj)
ss_x(obj, n(1:3), 1:ndim(obj))
# above is equivalent to obj[1:3, 1:3, drop = FALSE]
ii_x(obj, i = \(x) x > 5)
# above is equivalent to obj[obj > 5]
ss_x(obj, n(c("a", "a")), 2L)
# above is equivalent to obj[, lapply(c("a", "a"), \(i) which(colnames(obj) == i)) |> unlist()]

obj <- array(1:64, c(4,4,3))
print(obj)
ss_x(obj, n(1:3, 1:2), c(1,3))
# above is equivalent to obj[1:3, , 1:2, drop = FALSE]
ii_x(obj, i = \(x)x > 5)
# above is equivalent to obj[obj > 5]


################################################################################


# lists ====

obj <- list(a = 1:10, b = letters[1:11], c = 11:20)
print(obj)
ii_x(obj, 1) # obj[1]
```

```
ii_x(obj, 1:2) # obj[1:2]
ii_x(obj, is.numeric) # obj[sapply(obj, is.numeric)]
# for recursive subsets, see lst_rec()


obj <- rbind(
  lapply(1:4, \(x)sample(c(TRUE, FALSE, NA))),
  lapply(1:4, \(x)sample(1:10)),
  lapply(1:4, \(x)rnorm(10)),
  lapply(1:4, \(x)sample(letters))
)
colnames(obj) <- c("a", "b", "c", "a")
print(obj)
ss_x(obj, n(1:3), 1:ndim(obj))
# above is equivalent to obj[1:3, 1:3, drop = FALSE]
ii_x(obj, i = is.numeric)
# above is equivalent to obj[sapply(obj, is.numeric)]
ss_x(obj, n(c("a", "a")), 2L)
# above is equivalent to obj[, lapply(c("a", "a"), \(i) which(colnames(obj) == i)) |> unlist()]

obj <- array(as.list(1:64), c(4,4,3))
print(obj)
ss_x(obj, n(1:3, 1:2), c(1,3))
# above is equivalent to obj[1:3, , 1:2, drop = FALSE]
ii_x(obj, i = \(x)x > 5)
# above is equivalent to obj[sapply(obj, \(x) x > 5)]

################################################################################

# data.frame-like objects ====

obj <- data.frame(a = 1:10, b = letters[1:10], c = 11:20, d = factor(letters[1:10]))
print(obj)
tt_x(obj, 1:3, 1:3)
tt_x(obj, with(obj,  (a > 5) & (c < 19)), is.numeric)
```

---

indx_x *Exported Utilities*

Description

Exported utilities.
Usually the user won't need these functions.

Usage

```
indx_x(i, x, xnames, xsize)

indx_wo(i, x, xnames, xsize)

.is.0(x)
```

Arguments

i	See squarebrackets_index_args.
x	a vector, vector-like object, factor, data.frame, data.frame-like object, or a list.
xnames	names or dimension names
xsize	length or dimension size

Value

The subsetted object.

Examples

```
x <- 1:10
names(x) <- letters[1:10]
indx_x(1:5, x, names(x), length(x))
indx_wo(1:5, x, names(x), length(x))
```

---

is.formula *Check if an Object is a Formula*

Description

`is.formula()` checks if an object is a formula.

Usage

```
is.formula(form)
```

Arguments

form	object to check

Value

The `is_formula()` function returns `TRUE` if the input is a formula, and `FALSE` otherwise.

Examples

```
is.formula(~ x)
is.formula(1:10)
```

long *Index-less Subset Methods on (Long) Vectors*

Description

The long_ - methods are similar to the ii_ - methods, except they don't require an indexing vector, and are designed for memory efficiency.

Usage

```
long_x(x, ...)

## Default S3 method:
long_x(
  x,
  stride,
  ...,
  use.names = TRUE,
  sticky = getOption("squarebrackets.sticky", FALSE)
)

long_set(x, ...)

## Default S3 method:
long_set(x, stride, ..., rp, tf)
```

Arguments

x	an atomic object. For long_x(), couldb.mutatomic(x) must be TRUE. For long_set() it must be a mutatomic **variable**.
...	see squarebrackets_ellipsis.
stride	see squarebrackets_stride.
use.names	Boolean, indicating if flat names should be preserved. Note that, since the long_ methods operates on **virtual** interior indices of an array/vector only, dimensions and dimnames are always dropped. for performance: set to FALSE
sticky	see squarebrackets_options.
rp, tf	see squarebrackets_modify.

Value

For long_x(): returns the sub-setted object.
Fr long_set(): returns nothing, but modifies the object by reference.

Examples

```
# extract all elements of x with the name "a":
nms <- c(letters, LETTERS, month.abb, month.name) |> rep_len(1e6)
x <- mutatomic(1:1e6, names = nms)
head(x)
stride <-  stride_v(names(x), v = "a")
long_x(x, stride) |> head()


# find all x smaller than or equal to 5, and replace with `-1000`:
stride <- stride_v(x, v = c(-Inf, 5))
long_set(x, stride, rp = -1000L)
head(x, n = 10)




x <- mutatomic(1:1e7)

# extract elements 2 to 9
long_x(x, ~ 2:9:1:1)

# reverse:
long_x(x, ~ 9:2:1:1)

# remove all elements except the last 10:
long_x(x, ~ 1:(.N - 10):1:-1)

# replace every other element:
x <- mutatomic(1:1e7)
long_set(x, ~ 2:.N:2:1, rp = -1)
head(x)

# replace all elements except the first element:
x <- mutatomic(1:1e7)
long_set(x, ~1:1:1:-1, rp = -1)
head(x)


# extract pattern c(TRUE, FALSE, FALSE, TRUE) from first 20 elements:
x <- mutatomic(1:1e7)
long_x(x, ~ 1:20:c(TRUE, FALSE, FALSE, TRUE):1)
```

`lst_rec` *Access, Replace, Transform, Delete, or Extend Recursive Subsets*

Description

The `lst_rec()` and `lst_recin()` methods are essentially convenient wrappers around `[[` and `[[<-`, respectively.

`lst_rec()` will access recursive subsets of lists.

`lst_recin()` can do the following things:

- replace or transform recursive subsets of a list, using R's default Copy-On-Modify semantics, by specifying the `rp` or `tf` argument, respectively.
- delete a recursive subset of a list, using R's default Copy-On-Modify semantics, by specifying argument `rp = NULL`.
- extending a list with additional recursive elements, using R's default Copy-On-Modify semantics.
  This is done by specifying an out-of-bounds index in argument `rec`, and entering the new values in argument `rp`.
  Note that adding surface level elements of a dimensional list will delete the dimension attributes of that list.

Usage

```
lst_rec(x, ...)

## Default S3 method:
lst_rec(x, rec, ...)

lst_recin(x, ...)

## Default S3 method:
lst_recin(x, rec, ..., rp, tf)
```

Arguments

`x`	a list, or list-like object.
`...`	see squarebrackets_ellipsis.
`rec`	a strictly positive integer vector or character vector, of length `p`, such that `lst_rec(x, rec)` is equivalent to `x[[ rec[1] ]]...[[ rec[p] ]]`, providing all but the final indexing results in a list. When on a certain subset level of a nested list, multiple subsets with the same name exist, only the first one will be selected when performing recursive indexing by name, since recursive indexing can only select a single element. `NA, NaN, Inf, -Inf, NULL` are not valid values for `rec`.
`rp`	optional, and allows for multiple functionalities: • In the simplest case, performs `x[[rec]] <- rp`, using R's default semantics. Since this is a replacement of a recursive subset, `rp` does not necessarily have to be a list itself; `rp` can be any type of object. • Specifying `rp = NULL` will **delete** (recursive) subset `lst_rec(x, rec)`. To specify actual `NULL` instead of deleting a subset, use `rp = list(NULL)`. • When `rec` is an integer, and specifies an out-of-bounds subset, `lst_recin()` will add value `rp` to the list. Any empty positions in between will be filled with `NA`. • When `rec` is character, and specifies a non-existing name, `lst_recin()` will add value `rp` to the list as a new element at the end.

`tf` an optional function. If specified, performs `x[[rec]] <- tf(x[[rec]])`, using R's default Copy-On-Modify semantics.
Does not support extending a list like argument `rp`.

Details

Since recursive objects are references to other objects, extending a list or deleting an element of a list does not copy the entire list, in contrast to atomic vectors.

Tip:
Dimensional sub-set operations on dimensional lists are much faster and more flexible than Recursive sub-set operations on nested lists.
So, whenever it makes sense, consider turning your nested list into a dimensional list.
One can turn a hierarchical list into a dimensional list using, for example, the broadcast::cast_hier2dim method from the 'broadcast' 'R' package.

Value

For `lst_rec()`:
Returns the recursive subset.

For `lst_recin(..., rp = rp)`:
Returns VOID, but replaces, adds, or deletes the specified recursive subset, using R's default Copy-On-Modify semantics.

For `lst_recin(..., tf = tf)`:
Returns VOID, but transforms the specified recursive subset, using R's default Copy-On-Modify semantics.

Examples

```
lst <- list(
  A = list(
    A = list(A = "AAA", B = "AAB"),
    A = list(A  = "AA2A", B = "AA2B"),
    B = list(A = "ABA", B = "ABB")
  ),
  B = list(
    A = list(A = "BAA", B = "BAB"),
    B = list(A = "BBA", B = "BBB")
  ),
  C = list(
    A = 1:10,
    B = 11:20
  )
)

################################################################################

# access recursive subsets ====
```

```
lst_rec(lst, c(1,2,2)) # this gives "AA2B"
lst_rec(lst, c("A", "B", "B")) # this gives "ABB"
lst_rec(lst, c(2,2,1)) # this gives "BBA"
lst_rec(lst, c("B", "B", "A")) # this gives "BBA"


################################################################################

# replace recursive subset with R's default in-place semantics ====

# replace "AAB" using R's default in-place semantics:
lst_recin(
  lst, c("A", "A", "B"),
  rp = "THIS IS REPLACED WITH IN-PLACE SEMANTICS"
)
print(lst)



################################################################################

# transform recursive subsets with R's default in-place semantics ====

lst_recin(lst, c("C", "A"), tf = \(x)x^2) # transforms lst$C$A

print(lst)


################################################################################

# add/remove new recursive subsets with R's default in-place semantics ====

lst_recin(lst, c("C", "D"), rp = "NEW VALUE") # adds lst$C$D
print(lst)

lst_recin(lst, c("C", "A"), rp = NULL) # removes lst$C$A
print(lst) # notice lst$C$A is GONE


################################################################################

# Modify View of List By Reference ====

x <- list(
 a = data.table::data.table(cola = 1:10, colb = letters[1:10]),
 b = data.table::data.table(cola = 11:20, colb = letters[11:20])
)
print(x)
myref <- lst_rec(x, "a")
address(myref) == address(x$a) # they are the same
tt_set(myref, col = "cola", tf = \(x)x^2)
print(x) # notice x has been changed
```

match_all *Match All, Order-Sensitive and Duplicates-Sensitive*

Description

Find all indices of vector haystack that are equal to vector needles, taking into account the order of both vectors, and their duplicate values.

match_all() is essentially a much more efficient version of:

```
lapply(needles, \(i) which(haystack == i))
```

Like lapply(needles, \(i) which(haystack == i)), NAs are ignored.

match_all() internally calls collapse::fmatch and collapse::gsplit.
Core of the code is based on a suggestion by Sebastian Kranz (author of the 'collapse' package).

Usage

```
match_all(needles, haystack, unlist = TRUE)
```

Arguments

needles, haystack	vectors of the same type. needles cannot contain NA/NaN. Long vectors are not supported.
unlist	Boolean, indicating if the result should be a single unnamed integer vector (TRUE, default), or a named list of integer vectors (FALSE).

Value

An integer vector, or list of integer vectors.
If a list, each element of the list corresponds to each value of needles.
When needles and/or haystack is empty, or when haystack is fully NA, match_all() returns an empty integer vector (if unlist = TRUE), or an empty list (if unlist = FALSE).

Examples

```
n <- 200
haystack <- sample(letters, n, TRUE)
needles <- sample(letters, n/2, TRUE)
indices1 <- match_all(needles, haystack)
head(indices1)
```

mutatomic_class *A Class of Mutable Atomic Array-Like Objects*

Description

The mutatomic class is a class of mutable atomic array-like objects.
It works exactly the same in all aspects as regular atomic vectors and arrays.
There is only one real difference:
Pass-by-reference functions in 'squarebrackets' only accept atomic objects when they are of class mutatomic, for greater safety.
In all other aspects, mutatomic objects are the same as R's regular atomic objects, including the behaviour of the [<- operator .

Exposed functions (beside the S3 methods):

- mutatomic(): create a mutatomic object from given data.
- couldb.mutatomic(): checks if an object could become mutatomic.

The mutatomic class is a class of mutable atomic array-like objects.
It works exactly the same in all aspects as regular atomic vectors and arrays.
There is only one real difference:
Pass-by-reference functions in 'squarebrackets' only accept atomic objects when they are of class mutatomic, for greater safety.
In all other aspects, mutatomic objects are the same as R's regular atomic objects, including the behaviour of the [<- operator .

Exposed functions (beside the S3 methods):

- mutatomic(): create a mutatomic object from given data.
- couldb.mutatomic(): checks if an object could become mutatomic.

Usage

```
mutatomic(data, names = NULL, dim = NULL, dimnames = NULL, comment = NULL)

as.mutatomic(x, ...)

is.mutatomic(x)

couldb.mutatomic(x)
```

Arguments

data	atomic vector giving data to fill the mutatomic object.
names, dim, dimnames, comment	see their respective help pages.
x	an atomic object.
...	method dependent arguments.

Details

`couldb.mutatomic()`:
The `couldb.mutatomic()` function checks if an object can be converted to a `mutatomic` object.
Only objects with the following properties can be converted to `mutatomic`:

- `atomic` **data type**:
  The data type of the object is `atomic`.
  I.e. raw, logical, integer, double, complex, or character.
  Thus recursive objects, like lists, and S4 objects, and so on cannot become `mutatomic`.
  `ALTREP` objects will be materialized when coerced to `mutatomic`.
- **length equals number of elements**:
  The length of the object equals the number of elements.
  This is the case for the vast majority of objects.
  But some objects, like the various classes of the 'bit' package, break this principle and are thus not compatible with `mutatomic`.
- **product of dimensions equals number of elements**:
  If the object has dimensions, the product of the dimension sizes must equal the number of elements.
- **array-like**:
  The object is "array-like", meaning that the object is a vector where one can *meaningfully* set, remove, or modify the dimensions, without breaking its functionality.
  Thus a factor (which already represents a dummy matrix), most data-time objects (which generally do not support dimensions), and special printed vectors (hexmode, octmode, romans), cannot be `mutatomic`.
- **missing values consistent with base 'R'**:
  The meaning of `NA` in the vector is consistent with that of base 'R'.
  This is the case for the vast majority of objects.
  But some classes, like `integer64` has a different definition of `NA` than base 'R', which is not supported by the `mutatomic` class.
- **not an S4 class**:
  The object is not an S4 class.

`couldb.mutatomic()`:
The `couldb.mutatomic()` function checks if an object can be converted to a `mutatomic` object.
Only objects with the following properties can be converted to `mutatomic`:

- `atomic` **data type**:
  The data type of the object is `atomic`.
  I.e. raw, logical, integer, double, complex, or character.
  Thus recursive objects, like lists, and S4 objects, and so on cannot become `mutatomic`.
  `ALTREP` objects will be materialized when coerced to `mutatomic`.
- **length equals number of elements**:
  The length of the object equals the number of elements.
  This is the case for the vast majority of objects.
  But some objects, like the various classes of the 'bit' package, break this principle and are thus not compatible with `mutatomic`.

- **product of dimensions equals number of elements**:
  If the object has dimensions, the product of the dimension sizes must equal the number of elements.
- **array-like**:
  The object is "array-like", meaning that the object is a vector where one can *meaningfully* set, remove, or modify the dimensions, without breaking its functionality.
  Thus a factor (which already represents a dummy matrix), most data-time objects (which generally do not support dimensions), and special printed vectors (hexmode, octmode, romans), cannot be `mutatomic`.
- **missing values consistent with base 'R'**:
  The meaning of `NA` in the vector is consistent with that of base 'R'.
  This is the case for the vast majority of objects.
  But some classes, like `integer64` has a different definition of `NA` than base 'R', which is not supported by the `mutatomic` class.
- **not an S4 class**:
  The object is not an S4 class.

Value

For `mutatomic()`, `as.mutatomic()`:
Returns a `mutatomic` object.

For `is.mutatomic()`:
Returns `TRUE` if the object is `mutatomic`, and returns `FALSE` otherwise.

For `couldb.mutatomic()`:
Returns `TRUE` if the object can be converted to `mutatomic`.
Returns `FALSE` otherwise.

For `mutatomic()`, `as.mutatomic()`:
Returns a `mutatomic` object.

For `is.mutatomic()`:
Returns `TRUE` if the object is `mutatomic`, and returns `FALSE` otherwise.

For `couldb.mutatomic()`:
Returns `TRUE` if the object can be converted to `mutatomic`.
Returns `FALSE` otherwise.

Footnotes

- Always use the exported functions given by 'squarebrackets' to create a `mutatomic` object, as they make necessary safety checks.
- Mutable objects that can only exist as vectors can be modified via functions from the 'data.table' or 'collapse' packages.
- Mutable date-time objects already exist in the form of S4 classes, and need no coverage from the 'mutatomic' class.

- While one can force dimension attributes upon factors and base R's date-time objects, this may break or corrupt it's interactions with methods (like the `[` and `print()` methods), coercions (like `as.data.frame()`), and array manipulators (like `aperm()`).
  'squarebrackets' does not allow such sloppiness, and thus strictly consider factors and date-time objects to be vectors but not 'array-like'.

- It would not make much sense to make factors dimensional, since they are primarily used to represent a dummy matrix for statistical modelling.
  A dimensional array of categorical values that's not used for modelling is better off being represented as either character array or integer array.

- Always use the exported functions given by 'squarebrackets' to create a `mutatomic` object, as they make necessary safety checks.
- Mutable objects that can only exist as vectors can be modified via functions from the 'data.table' or 'collapse' packages.
- Mutable date-time objects already exist in the form of S4 classes, and need no coverage from the 'mutatomic' class.
- While one can force dimension attributes upon factors and base R's date-time objects, this may break or corrupt it's interactions with methods (like the `[` and `print()` methods), coercions (like `as.data.frame()`), and array manipulators (like `aperm()`).
  'squarebrackets' does not allow such sloppiness, and thus strictly consider factors and date-time objects to be vectors but not 'array-like'.

- It would not make much sense to make factors dimensional, since they are primarily used to represent a dummy matrix for statistical modelling.
  A dimensional array of categorical values that's not used for modelling is better off being represented as either character array or integer array.

Examples

```
x <- mutatomic(
  1:20, dim = c(5, 4), dimnames = list(letters[1:5], letters[1:4])
)
x

x <- matrix(1:10, ncol = 2)
x <- as.mutatomic(x)
is.mutatomic(x)
print(x)
x[, 1]
x[] <- as.double(x)
print(x)
is.mutatomic(x)


x <- mutatomic(
  1:20, dim = c(5, 4), dimnames = list(letters[1:5], letters[1:4])
)
x
```

```
x <- matrix(1:10, ncol = 2)
x <- as.mutatomic(x)
is.mutatomic(x)
print(x)
x[, 1]
x[] <- as.double(x)
print(x)
is.mutatomic(x)
```

`n` *Nest*

Description

The c() function concatenates vectors or lists into a vector (if possible) or else a list.
In analogy to that function, the `n()` function **nests** objects into a list (not into an atomic vector, as atomic vectors cannot be nested).
It is a short-hand version of the list function.
This is handy because lists are often needed in 'squarebrackets', especially for arrays.

Usage

```
n()
```

Value

The list.

Examples

```
obj <- array(1:64, c(4,4,3))
print(obj)
ss_x(obj, n(1:3, 1:2), c(1,3))
# above is equivalent to obj[1:3, , 1:2, drop = FALSE]
```

`ndim` *Get Size Properties of an Object*

Description

`ndim(x)` is short-hand for `length(dim(x))`.

`s(x, m)` gets the size of an object `x` along margin `m`.
If `m == 0`, `s()` gets the length of an object.
If `m > 0`, `s()` gets `dim(x)[m]` of the object.

Usage

```
ndim(x)

rdim(x)

s(x, ...)

## Default S3 method:
s(x, m = if (is.null(dim(x))) 0L else dim(x), ...)
```

Arguments

x	a vector, array, or data.frame.
...	further arguments passed to or from methods.
m	the margin.

Value

For ndim():
An integer, giving the number of dimensions x has.
For vectors, gives 0L.

For rdim():
An integer vector, giving the range 1:ndim(x).
For vectors, gives 0L.

For s():
A numeric vector giving the size(s) of the object at the given margin(s).

Examples

```
x <- 1:10
ndim(x)
obj <- array(1:64, c(4,4,3))
print(obj)
ndim(obj)
```

---

sb_setRename *Safely Change the Names of a Mutable Object By Reference*

Description

Functions to rename a supported mutable object using pass-by-reference semantics:

- sb_setFlatnames() renames the (flat) names of a mutatomic object.
- sb_setDimnames() renames the dimension names of a mutatomic object.
- sb_setVarnames() renames the variable names of a data.table object.

Usage

```
sb_setFlatnames(x, i = NULL, newnames, ...)

sb_setDimnames(x, m, newdimnames, ...)

sb_setVarnames(x, old, new, skip_absent = FALSE, ...)
```

Arguments

x	a **variable** belonging to one of the supported mutable classes.
i	logical, numeric, character, or imaginary indices, indicating which flatnames should be changed. If i = NULL, the names will be completely replaced.
newnames	Atomic character vector giving the new names. Specifying NULL will remove the names.
...	see squarebrackets_ellipsis.
m	integer vector giving the margin(s) for which to change the names (m = 1L for rows, m = 2L for columns, etc.).
newdimnames	a list of the same length as m. The first element of the list corresponds to margin m[1], the second element to m[2], and so on. The components of the list can be either NULL, or a character vector with the same length as the corresponding dimension. Instead of a list, simply NULL can be specified, which will remove the dimnames completely.
old	the old column names
new	the new column names, in the same order as old
skip_absent	Skip items in old that are missing (i.e. absent) in names(x). Default FALSE halts with error if any are missing.

Value

Returns: VOID. This method modifies the object by reference.
Do not use assignment like names(x) <- sb_setRename(x, ...).
Since this function returns void, you'll just get NULL.

Examples

```
# mutable atomic vector ====
x <- y <- mutatomic(1:10, names = letters[1:10])
print(x)
sb_setFlatnames(x, newnames = rev(letters[1:10]))
print(y)

x <- y <- mutatomic(1:10, names = letters[1:10])
```

```
print(x)
sb_setFlatnames(x, 1L, "XXX")
print(y)

################################################################################


# mutable atomic matrix ====
x <- mutatomic(
  1:20, dim = c(5, 4), dimnames = n(letters[1:5], letters[1:4])
)
print(x)
sb_setDimnames(
  x,
  1:2,
  lapply(dimnames(x), rev)
)
print(x)



################################################################################



# data.table ====

x <- data.table::data.table(
  a = 1:20,
  b = letters[1:20]
)
print(x)
sb_setVarnames(x, old = names(x), new = rev(names(x)))
print(x)
```

`ss2ii` *Convert Subscripts to Coordinates, Coordinates to Interior Indices, and Vice-Versa*

Description

These functions convert a list of integer subscripts to an integer matrix of coordinates, an integer matrix of coordinates to an integer vector of interior indices, and vice-versa.
Inspired by the `sub2ind` function from 'MatLab'.

- `ss2coord()` converts a list of integer subscripts to an integer matrix of coordinates.
- `coord2ii()` converts an integer matrix of coordinates to an integer vector of interior indices.
- `ii2coord()` converts an integer vector of interior indices to an integer matrix of coordinates.
- `coord2ss()` converts an integer matrix of coordinates to a list of integer subscripts;
  it performs a **very naive** conversion.

- `ss2ii()` is a faster and more memory efficient version of `ss2coord(sub, x.dims) |> coord2ii(x.dims)`

All of these functions are written to be memory-efficient.
The `coord2ii()` is thus the opposite of arrayInd, and `ii2coord` is merely a convenient wrapper around arrayInd.

Usage

```
ss2coord(sub, x.dim)

coord2ss(coord)

coord2ii(coord, x.dim, checks = TRUE)

ii2coord(ind, x.dim)

ss2ii(sub, x.dim, checks = TRUE)
```

Arguments

`sub`	a list of integer subscripts. The first element of the list corresponds to the first dimension (rows), the second element to the second dimensions (columns), etc. The length of `sub` must be equal to the length of `x.dim`. One cannot give an empty subscript; instead fill in something like `seq_len(dim(x)[margin])`. NOTE: The `coord2ss()` function does not support duplicate subscripts.
`x.dim`	an integer vector giving the dimensions of the array in question. I.e. `dim(x)`.
`coord`	an integer matrix, giving the coordinate indices (subscripts) to convert. Each row is an index, and each column is the dimension. The first columns corresponds to the first dimension, the second column to the second dimensions, etc. The number of columns of `coord` must be equal to the length of `x.dim`.
`checks`	Boolean, indicating if arguments checks should be performed. Defaults to `TRUE`. Can be set to `FALSE` for minor speed improvements. for performance: set to FALSE
`ind`	an integer vector, giving the interior position indices to convert.

Details

The base S3 vector and array classes in 'R' use the standard Linear Algebraic convention, as in academic fields like Mathematics and Statistics, in the following sense:

- vectors are **column** vectors (i.e. vertically aligned vectors);

- index counting starts at 1;
- rows are the first dimension/subscript, columns are the second dimension/subscript, etc.

Thus, the orientation of interior indices in, for example, a 4-rows-by-5-columns matrix, is as follows:

```
     [,1] [,2] [,3] [,4] [,5]
[1,]    1    5    9   13   17
[2,]    2    6   10   14   18
[3,]    3    7   11   15   19
[4,]    4    8   12   16   20
```

So in a 4 by 5 matrix, subscript `[1, 2]` corresponds to interior index 5.
Array subscripting in 'squarebrackets' also follows this standard convention.

Value

For `ss2coord()` and `ii2coord()`:
Returns an integer matrix of coordinates (with properties as described in argument `coord`).

For `coord2ii()`:
Returns an numeric vector of interior indices (with properties as described in argument `ind`).

For `coord2ss()`:
Returns a list of integer subscripts (with properties as described in argument `sub`)

For `ss2ii()`:
Returns an integer vector of interior indices(if `prod(x.dim) < (2^31 - 1)`), or an numeric vector of interior indices (if `prod(x.dim) >= (2^31 - 1)`).

Note

These functions were not specifically designed for duplicate indices per-sé.
For efficiency, they do not check for duplicate indices either.

Examples

```
x.dim <- c(10, 10, 3)
x.len <- prod(x.dim)
x <- array(1:x.len, x.dim)
sub <- list(c(4, 3), c(3, 2), c(2, 3))
coord <- ss2coord(sub, x.dim)
print(coord)
ind <- coord2ii(coord, x.dim)
print(ind)
all(x[ind] == c(x[c(4, 3), c(3, 2), c(2, 3)])) # TRUE
coord2 <- ii2coord(ind, x.dim)
print(coord)
all(coord == coord2) # TRUE
```

```
sub2 <- coord2ss(coord2)
sapply(1:3, \(i) sub2[[i]] == sub[[i]]) |> all() # TRUE
```

`stopifnot_ma_safe2mutate`
Developer Functions for the mutatomic Class

Description

The `stopifnot_ma_safe2mutate()` function checks if an atomic object is actually safe to mutate. The `.internal_set_ma()` function sets an object to class 'mutatomic' by reference.

Usage

```
stopifnot_ma_safe2mutate(sym, envir, .abortcall)

address(x)

.internal_set_ma(x)
```

Arguments

`sym`	the symbol of the object; i.e. `substitute(x)`.
`envir`	the environment where the object resides; i.e. `parent.frame(n = 1)`.
`.abortcall`	environment where the error message is passed to.
`x`	atomic object

Value

Nothing. Only gives an error if the object is not safe to mutate.

Examples

```
testfun1 <- function(x) {
  .internal_set_ma(x)
}


x <- sample(1:10)
is.mutatomic(x)

testfun1(x)
is.mutatomic(x)
print(x)
```

stride_eval *Evaluate Stride Object*

Description

`eval_stride()` evaluates a stride or formula object.
This can be handy to check exactly what parameters will be fed to the `long_` methods.

`formula2stride()` converts a formula to a stride_seq or stride_ptrn object.

Usage

```
eval_stride(stride, x)

## Default S3 method:
eval_stride(stride, x)

is.stride(x)

formula2stride(form, x)
```

Arguments

`stride`	see squarebrackets_stride.
`x`	a (long) atomic vector.
`form`	a formula, as described in squarebrackets_stride.

Value

Using `stride_v()`
The original stride object, but as a list.

Using `stride_seq()` **or** `stride_ptr()`
A list with at least the following elements:

`$start`:
The actual starting point of the sequence.
This is simply `from` translated to regular numeric.

`$end`:
The **actual** ending point of the sequence.
This is not the same as `to`.
For example, the following code:

```
seq(from = 1L, to = 10L, by = 2L)
#> [1] 1 3 5 7 9
```

specifies `to = 10L`.
But the sequence doesn't actually end at `10`; it ends at `9`.

Therefore, `stride_seq(x, 1, 10, 2) |> eval_stride()` will return `end = 9`, not `end = 10`.
This allows the user to easily predict where an sequence given in stride_seq/stride_ptrn will actually end.

`$len`:
The actual vector length the sequence would be, given the translated parameters.

Examples

```
# extract all elements of x with the name "a":
nms <- c(letters, LETTERS, month.abb, month.name) |> rep_len(1e6)
x <- mutatomic(1:1e6, names = nms)
head(x)
stride <-  stride_v(names(x), v = "a")
long_x(x, stride) |> head()


# find all x smaller than or equal to 5, and replace with `-1000`:
stride <- stride_v(x, v = c(-Inf, 5))
long_set(x, stride, rp = -1000L)
head(x, n = 10)




x <- mutatomic(1:1e7)

# extract elements 2 to 9
long_x(x, ~ 2:9:1:1)

# reverse:
long_x(x, ~ 9:2:1:1)

# remove all elements except the last 10:
long_x(x, ~ 1:(.N - 10):1:-1)

# replace every other element:
x <- mutatomic(1:1e7)
long_set(x, ~ 2:.N:2:1, rp = -1)
head(x)

# replace all elements except the first element:
x <- mutatomic(1:1e7)
long_set(x, ~1:1:1:-1, rp = -1)
head(x)


# extract pattern c(TRUE, FALSE, FALSE, TRUE) from first 20 elements:
x <- mutatomic(1:1e7)
long_x(x, ~ 1:20:c(TRUE, FALSE, FALSE, TRUE):1)
```

`stride_ptrn` *Pattern Sequence-based stride*

Description

`stride_ptrn()` specifies a patterned sequence, in the form of `(start:end)[ptrn]`, for use in the long_ methods.

For example:
the sub-set operation `long_x(x, stride_ptrn(start, end, ptrn))`
is conceptually equivalent to
`x[(start:end)[ptrn]]`

`ptrn_len(start, end, ptrn)` calculates the length of `(start:end)[ptrn]`, without actually allocating said vector.

Usage

```
stride_ptrn(start, end, ptrn, use = 1L)

ptrn_len(start, end, ptrn)
```

Arguments

`start`	natural scalar giving the starting point of the sequence.
`end`	natural scalar giving the ending point of the sequence.
`ptrn`	a logical vector, giving the pattern. `ptrn` must contain at least 1 `TRUE` and 1 `FALSE` element, to avoid ambiguity. `ptrn` must have length that fulfils all of the following conditions: • it is `>= 2` • it is `<= length(start:end)` • it is `<= 2^31 - 1`
`use`	1 to select indices `(start:end)[ptrn]`; `-1` to select all indices **except** `(start:end)[ptrn]`

Value

An object of class "stride".

See Also

squarebrackets_stride

Examples

```
x <- mutatomic(1:1e7)

long_x(x, stride_ptrn(2, 20, c(TRUE, FALSE, FALSE, TRUE)))

long_x(x, stride_ptrn(20, 2, c(TRUE, FALSE, FALSE, TRUE)))

long_x(x, stride_ptrn(2, 20, c(TRUE, FALSE, FALSE, TRUE), -1)) |> head()

long_x(x, stride_ptrn(20, 2, c(TRUE, FALSE, FALSE, TRUE), -1)) |> head()
```

---

stride_seq *Regular Sequence-based stride*

Description

`stride_seq()` specifies a regular sequence for use in the `long_` methods.
For example:
`long_x(x, stride_seq(from, to, by))`
is conceptually equivalent to
`x[seq(from, to, by)]`

Usage

```
stride_seq(from, to, by, use = 1L)
```

Arguments

`from`	natural scalar giving the starting point of the sequence.
`to`	natural scalar giving the maximalle allowed end value.
`by`	natural scalar giving the step-size.
`use`	1 to select indices `seq(from, to, end)`; `-1` to select all indices **except** `seq(from, to, end)`

Value

An object of class "stride".

See Also

squarebrackets_stride

Examples

```
x <- mutatomic(1:1e7)

# extract elements 2 to 9
long_x(x, stride_seq(2, 9, 1))

# reverse:
long_x(x, stride_seq(9, 2, 1))

# remove elements 1 to length(x) - 10:
long_x(x, stride_seq(1, length(x) - 10, 1, -1))

# replace every other element:
x <- mutatomic(1:1e7)
long_set(x, stride_seq(2, length(x), 2), rp = -1)
head(x)

# replace all elements except the first element:
x <- mutatomic(1:1e7)
long_set(x, stride_seq(1, 1, 1, -1), rp = -10)
head(x)
```

---

stride_v *Value-based stride*

Description

`stride_v()` is used in the long_ methods to specify sub-set operations based on values in an atomic vector of properties.

`stride_v()` can be used in the long_ methods to perform value-based sub-setting operations.
For a very basic understanding, consider the following illustration.
In the simplest terms, the sub-set operation `long_x(x, stride_v(y, v = v, na = na))`
is conceptually equivalent to the following:

```
x[ifelse(is.na(y), na, y == v)] # if `na` is `TRUE` or `FALSE`
x[is.na(y)] # if `na` is `NA`
```

Usage

```
stride_v(y, ...)

## Default S3 method:
stride_v(y, ..., v = NULL, na = FALSE, use = 1)
```

Arguments

y	a vector, with the same length as x, and ideally related to x For example, y may be the character vector names(x), the raw vector broadcast::checkNA(x, "raw"), the classless (i.e. raw data) values of x, or even the raw data values of another long vector with the same length as x. Note that, in the default method, couldb.mutatomic(y) must be TRUE, otherwise an error is returned.
...	not supported: all arguments in stride_v() other than y must be explicitly named.
v	a scalar or vector, depending on the type of y, indicating what values in y to look for. Details are given in the sections below.
na	TRUE, FALSE, or NA, indicating what to do with NAs/NaNs. If na = TRUE, NAs/NaNs are included in the sub-set operation (i.e. NAs/NaNs are extracted, removed, replaced, etc.). If na = FALSE, NAs/NaNs are excluded from the sub-set operation (i.e. NAs/NaNs are not extracted, not removed, not replaced, etc.). If na = NA, v is ignored, and **only** NA values are searched for the sub-set operation. See also the additional sections below.
use	1 to check for specified condition, -1 to check for the negated condition (i.e. !condition).

Value

An object of class "stride".

The Basic Idea

The basic idea is as follows.
Let x and y be 2 atomic vectors of the same length (but they don't have to be of the same type).
Let v be some atomic scalar of the same type as y.
Given the result of the condition y == v, the basic idea is to perform the following sub-set operations:

```
long_x(x, stride_v(y, v = v))            # ==> x[y == v]
long_set(x, stride_v(y, v = v), rp = rp) # ==> x[y == v] <- rp
long_set(x, stride_v(y, v = v), tf = tf) # ==> x[y == v] <- tf(x[y == v])
```

The above is with the default argument specification use = 1.
Of course one can invert the relationship by specifying argument use = -1, to get something like the following:

```
long_x(x, stride_v(y, v = v, use = -1))             # ==> x[p != v]
long_set(x, stride_v(y, v = v, use = -1), rp = rp)  # ==> x[p != v] <- rp
long_set(x, stride_v(y, v = v, use = -1), tf = tf)  # ==> x[p != v] <- tf(x[p != v])
```

And `y` is allowed to be the same vector as `x`, of course.

This basic idea, however, can become more complicated, depending on the atomic type of `y`, which is discussed in the next section.

Details per Atomic Type

Logical, Raw, Complex

For `y` of type `logical`, `raw`, and `complex`, stride_v works exactly as explained in the previous section.
`y` and `v` must be of the same atomic type.

Numeric

For `y` of type `integer` or `double` (collectively referred to as "numeric"), the basic idea laid-out before still holds:
one can use atomic vector `y` and atomic scalar `v` to perform sub-set operations like
`x[y == v]`.

But one may be more interested in a range of numbers, rather than one specific number (especially considering things like measurement error, and machine precision, and greater-than/larger-than relationships).
So for numeric `y`, one can also supply `v` of length **2**.
When `length(v) == 2L`, `long_` will check whether `y` is inside (or outside if `use = -1`) the bounded range given by `v`.
I.e. :

```
y >= v[1] & y <= v[2]
```

Note that `y` and `v` must both be numeric here, but they don't have to be the same type.
I.e. one can have `y` of type `integer` and `v` of type `double`, without problems.

Character

For `y` of type `character`, the basic idea is still to do something like `x[y == v]`.

When searching for string `v` for sub-setting purposes, one may want to take into consideration things like different spelling, spacing, or even encodings of the same string.
Implementing every form of fuzzy matching or encoding matching is computationally intensive, and also quite beyond the scope of this package.
Instead, the user may supply a character vector `v` of arbitrary length, containing all the variations (in terms of spelling, spacing, encoding, or whatever) of all the strings to look for.

So if a vector is given for `v` (instead of a single string), the following check is performed:

```
y %in% v
```

NOTE
The order of v is **irrelevant**.

Smaller Than, Greater Than

For numeric y, one can specify a range for v, as explained earlier.
But note one can also specify something like `v = c(-Inf, 4)`, which essentially corresponds to the condition `y <= 4`.
Thus, when v specifies a range, "greater-than" and "smaller-than" comparisons are also possible.

Handling NAs and NaN

We also have to handle the NAs and NaNs.
The na argument can be used to specify what to do when a y is NA.

When `na = FALSE`, all NA values of y are always ignored.
I.e. `long_x(x, stride_v(y, v = v, na = FALSE), use = 1)` will not extract NAs/NaNs,
and `long_x(x, stride_v(y, v = v, na = FALSE, use = -1))` will not remove NAs/NaNs.

When `na = TRUE`, NA values of y are always included.
I.e. `long_x(x, stride_v(y, v = v, na = TRUE), use = 1)` will also extract NAs/NaNs,
and `long_x(x, stride_v(y, v = v, na = TRUE, use = -1))` will also remove NAs/NaNs.

One can also specify `na = NA`, which will ignore v completely, and explicitly look for NAs/NaNs in y instead - like so:

```
long_x(x, stride_v(y, na = NA))                          # ==> x[is.na(y)]
long_x(x, stride_v(y, na = NA, use = -1))               # ==> x[!is.na(y)]
long_set(x, stride_v(y, na = NA), rp = rp)               # ==> x[is.na(y)] <- rp
long_set(x, stride_v(y, na = NA, use = -1), rp = rp)  # ==> x[!is.na(y)] <- rp
long_set(x, stride_v(y, na = NA), tf = tf)         # ==> x[is.na(y)] <- tf(x[is.na(y)])
long_set(x, stride_v(y, na = NA, use = -1), tf = tf)  # ==> x[!is.na(y)] <- tf(x[!is.na(y)])
```

Handling NAs/NaNs works the same for all atomic types.
For y of type `complex`, a value `p[i]` is considered NA if `Re(y[i])` is NA/NaN and/or `Im(y[i])` is NA/NaN.

Argument v is never allowed to contain NA/NaN.

All in One

Combining all of the above, one can allocate indices in base 'R' to be equivalent to the virtual indices produced by `stride_v(y, v = v, na = na, use = use)`, with the following code:

```
# if `na = NA`:
ind <- which(is.na(y)) * sign(use)

# else if using scalar `v`:
ind <- which(ifelse(is.na(y), na, y == v)) * sign(use)

# else if using numeric range for `v`:
ind <- which(ifelse(is.na(y), na, y >= v[1] & y <= v[2])) * sign(use)

# else if using character vector for `v`:
ind <- which(ifelse(is.na(y), na, y %in% v)) * sign(use)
```

Technical Details

On a technical level, the `stride_v()` method does the following:

1. Determine if we go through vector `y` in one go, or in chunks.
   If `lenght(y) >= 2^16`, vector `y` will be dealt with in `ceiling(length(x)^0.1)` equal-sized (so far as possible) chunks.
   Otherwise, treat `y` in one go; i.e. in one chunk.
2. Make a list equal to number of chunks from step one.
3. For each chunk, count the number of condition matches (`count`), the location of the first (`first`) match, and location of last (`last`) match.
4. For each chunk, do the following.
   If `count` is equal to `last - first + 1`, fill in list element for this chunk with `NULL`.
   If `count <= 2`, fill in list element for this chunk with `NULL`.
   Otherwise, fill list with `last - first + 1` bits (not bytes), specifying bit `1` if there's a match and bit `0` if no match.
   The bits will be stored in 32 bit integers. Thus each integer holds 32 elements worth of bits.

In 'R', an expression like `x[y == v]` is internally translated to `x[which(y == v)]`.
This means, 'R' will store 32 bits per element for the logical vector `y == v`, and 64 bits per element for the numeric vector from `which(y == v)`.
`stride_v()` stores information about the matches as 1 bit per condition (instead of 32 bits per condition), and only for the regions (chunks) where there's a need to store such data.
And `long_x()`/ `long_set()` will never call `which()`.
As such, `stride_v()` will **guarantee** to be **at least** 32 times more memory efficient than the base 'R' approach.
And the whole `long_x(x, stride_v(...))` / `long_set(x, stride_v(...))` operation will in most practical cases use **hundreds of times** less memory than the base 'R' approach!

See Also

squarebrackets_stride

Examples

```
# basic idea ====
```

```
nms <- c(letters, LETTERS, month.abb, month.name) |> rep_len(1e6)
x <- mutatomic(1:1e6, names = nms)
head(x)

# extract all elements of x with the name "a":
stride <-  stride_v(names(x), v = "a")
long_x(x, stride) |> head()

# find all x smaller than or equal to 5, and replace with `-1000`:
stride <- stride_v(x, v = c(-Inf, 5))
long_set(x, stride, rp = -1000L)
head(x, n = 10)


################################################################################
# Numeric range ====
#
x <- mutatomic(1:1e6)
head(x)
stride <- stride_v(x, v = c(-Inf, 5))
long_x(x, stride) # x[x <= 5]


################################################################################
# Character ====
#
if(require(stringi)) {
  x <- stringi::stri_rand_shuffle(rep("hello", 1e5))
  head(x)
  stride <- stride_v(x, v = "hello")
  long_x(x, stride) |> head() # find "hello"

  # find 2 possible misspellings of "hello":
  stride <- stride_v(x, v = c("holle", "helol"))
  long_x(x, stride) |> head()

}
```

Index